

# 第 1 课 颜色识别

## 【本课目标】：

本课学习使用 openmv 实现识别颜色的功能

## 【课前准备】：

OpenMV、USB 数据线

## 【本课内容】：

注意：在本节的源码例程中会有两个源码文件：multi\_color\_detection.py 和 single\_color\_detection.py

multi\_color\_detection.py 是可以同时识别多种颜色

single\_color\_detection.py 是只能识别一种颜色

根据自己的需要来选择一个使用即可，当然也可以两个都试一下。

注意在使用代码之前需要先将阈值范围调整一下，在下面有具体调节阈值的方法。

## 一、代码

### 单颜色识别示例代码

```
1.  ....
2.  单种颜色识别
3.  每次识别一种颜色，这里有红绿蓝三种颜色可以选择，还可自行添加其他颜色阈值来识别其他颜色
4.  颜色阈值采集方法：
5.  方法一：
6.      1. 可以在‘工具’-->‘机器视觉’-->‘阈值编辑器’-->‘帧缓冲区’中采集，
7.      2. 拖动 6 个滑块来调整图像，直到要采集的颜色都呈白色，其他都呈黑色为止，
8.      3. 采集好之后将 LAB 阈值复制到代码中的阈值处使用即可。
9.  方法二：
10.     1. 可以在右侧‘帧缓冲区’和‘直方图’的‘LAB 色彩空间’配合采集
11.     2. 先在帧缓冲区中用鼠标左键框出要采集的颜色区域，
12.     3. 然后在直方图中的 LAB 色彩空间中，分别复制 L、A、B 的最小值和最大值到代码中的颜色阈值元组中使用即可
13.  ...
14.  import sensor, time
15.
16.  # 颜色阈值索引，可以改变索引值来识别不同颜色此处 0 为红色，1 为绿色，2 为蓝色
17.  threshold_index = 0
18.
19.  # 设置颜色阈值，此处设置红绿蓝三种颜色的阈值
20.  thresholds = [(35, 56, 96, 48, -81, 96), # 红色阈值
21.                (21, 96, -49, -33, 20, 47), # 绿色阈值
22.                (16, 46, 9, 31, -15, -115)] # 蓝色阈值
23.
24.  # 初始化函数
25.  def init_setup():
26.      global sensor, clock # 设置为全局变量
```

```

27.     sensor.reset()                                # 初始化感光元件
28.     sensor.set_pixformat(sensor.RGB565)           # 设置感光元件图像色彩格式为 RGB565 (RGB565 为彩图, GRAYSCALE 为灰度图)
29.     sensor.set_framesize(sensor.QVGA)             # 设置感光元件分辨率大小为 QVGA (QVGA 是 320x240)
30.     sensor.skip_frames(time=2000)                 # 等待 2 秒, 使以上的设置生效
31.     sensor.set_auto_gain(False)                   # 关闭自动增益。在颜色识别中, 需要关闭自动增益
32.     sensor.set_auto_whitebal(False)               # 关闭白平衡。白平衡是默认开启的, 在颜色识别中, 需要关闭白平衡
33.     clock = time.clock()                           # 创建时钟对象
34.
35. # 主函数
36. def main():
37.     while 1:                                       # 无限循环
38.         clock.tick()                             # 更新 FPS 帧率时钟
39.         img = sensor.snapshot()                   # 拍一张照片并返回图像
40.         # 循环 3 种颜色阈值, 在图像中查看是否有对应颜色并做一些操作, [thresholds[threshold_index]]为列表格式
41.         for blob in img.find_blobs([thresholds[threshold_index]], pixels_threshold=200, area_threshold=200):
42.             img.draw_rectangle(blob.rect())        # 将目标颜色画框
43.             img.draw_cross(blob.cx(), blob.cy())   # 在目标颜色中心位置画个十字
44.             print("FPS %f" % clock.fps())         # 打印帧率
45.
46. # 程序入口
47. if __name__ == '__main__':
48.     init_setup()                                  # 执行初始化函数
49.     try:                                           # 异常处理
50.         main()                                    # 执行主函数
51.     except:
52.         pass

```

## 多颜色同时识别示例代码

```

1.     ....
2.     多颜色同时识别
3.     红绿蓝三种颜色可以同时识别, 还可自行添加其他颜色阈值来识别其他颜色
4.     颜色阈值采集方法:
5.     方法一:
6.         1. 可以在‘工具’-->‘机器视觉’-->‘阈值编辑器’-->‘帧缓冲区’中采集,
7.         2. 拖动 6 个滑块来调整图像, 直到要采集的颜色都呈白色, 其他都呈黑色为止,
8.         3. 采集好之后将 LAB 阈值复制到代码中的阈值处使用即可。
9.     方法二:
10.        1. 可以在右侧‘帧缓冲区’和‘直方图’的‘LAB 色彩空间’配合采集
11.        2. 先在帧缓冲区中用鼠标左键框出要采集的颜色区域,
12.        3. 然后在直方图中的 LAB 色彩空间中, 分别复制 L、A、B 的最小值和最大值到代码中的颜色阈值元组中使用即可
13.        ...
14.     import sensor, time
15.
16.     # 设置颜色阈值, 此处设置红绿蓝三种颜色的阈值
17.     thresholds = [(0, 23, 50, 20, 29, 1), # 红色阈值
18.                   (21, 96, -49, -33, 20, 47), # 绿色阈值
19.                   (6, 13, -13, 11, -13, -97)] # 蓝色阈值
20.
21.     # 初始化函数

```

```

22. def init_setup():
23.     global sensor, clock          # 设置为全局变量
24.     sensor.reset()                # 初始化感光元件
25.     sensor.set_pixformat(sensor.RGB565)  # 设置感光元件图像色彩格式为 RGB565 (RGB565 为彩图, GRAYSCALE 为灰度图)
26.     sensor.set_framesize(sensor.QVGA)  # 设置感光元件分辨率大小为 QVGA (QVGA 是 320x240)
27.     sensor.skip_frames(10)          # 跳过一些帧, 使以上的设置生效
28.     sensor.set_auto_gain(False)      # 关闭自动增益。在颜色识别中, 需要关闭自动增益
29.     sensor.set_auto_whitebal(False)   # 关闭白平衡。白平衡是默认开启的, 在颜色识别中, 需要关闭白平衡
30.     clock = time.clock()            # 创建时钟对象
31.
32. # 主函数
33. def main():
34.     while 1:                        # 无限循环
35.         clock.tick()                # 更新 FPS 帧率时钟
36.         img = sensor.snapshot()      # 拍一张照片并返回图像
37.         # 循环 3 种颜色阈值, 在图像中查看是否有对应颜色并做一些操作, thresholds 需为列表格式
38.         for blob in img.find_blobs(thresholds, pixels_threshold=200, area_threshold=200):
39.             img.draw_rectangle(blob.rect())  # 将目标颜色画框
40.             img.draw_cross(blob.cx(), blob.cy()) # 在目标颜色中心位置画个十字
41.         print("FPS %f" % clock.fps()) # 打印帧率
42.
43. # 程序入口
44. if __name__ == '__main__':
45.     init_setup()                    # 执行初始化函数
46.     try:                             # 异常处理
47.         main()                      # 执行主函数
48.     except:
49.         pass

```

## 二、代码使用方法

1.在资料中的‘源码例程’文件夹中找到颜色识别文件夹，然后选择单颜色识别或多颜色识别代码在 openmv ide 中打开。此处以单颜色识别为例

2.替换颜色阈值。

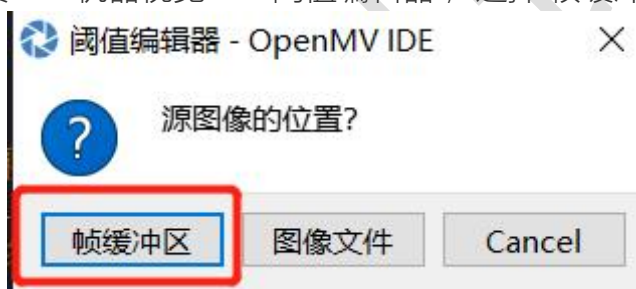
阈值查找方式这里主要介绍两种，都是在 openmv ide 中来操作的：

阈值查找方式一：

- (1) 在 openmv ide 中运行刚才打开的源码文件，
- (2) 然后将要识别颜色的物体放在摄像头画面内，(注意不要有相近颜色的物体干扰，否则效果不好)
- (3) 然后点击红色×号按钮停止运行程序，此时就可以在右上方的帧缓冲区中看到图像画面，



(4) 然后点击'工具'-->'机器视觉'-->'阈值编辑器', 选择'帧缓冲区'



(5) 看到如下界面



调整到如下图所示情况时即可复制 LAB 阈值到代码中使用了。

阈值编辑器 - OpenMV IDE

源图像

二进制图像 (白色像素是被跟踪的像素)

选择最佳的顏色跟踪閾值。

LAB

重置滑块

L最小值

L最大值

A最小值

A最大值

B最小值

B最大值

LAB閾值 (14, 49, 52, 94, 103, 31)

关闭前复制上面的閾值。

复制此LAB閾值，然后点击close，将复制的閾值替换到代码中

Close

```
13
14 import sensor, time
15
16 # 颜色阈值索引，可以改变索引值来识别不同颜色此处0为红色，1为绿
17 threshold_index = 0
18
19 # 设置颜色阈值，此外设置红绿蓝三种颜色的閾值
20 thresholds = [(14, 49, 52, 94, 103, 31) # 红色閾值
21               (21, 96, -49, -33, 20, 47), # 绿色閾值
22               (16, 46, 9, 31, -15, -115)] # 蓝色閾值
23
```

### 閾值查找方式二：

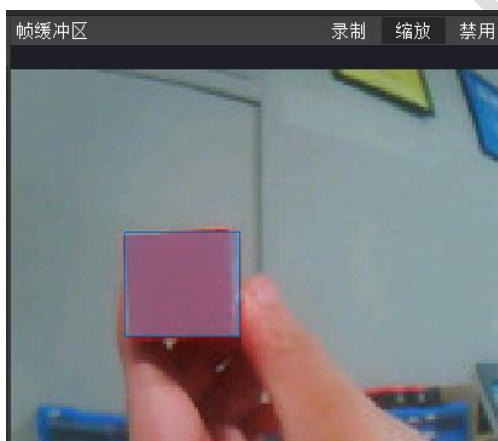
- (1) 在 openmv ide 中运行刚才打开的源码文件，
- (2) 然后将要识别颜色的物体放在摄像头画面内，(注意不要有相近颜色的物体干扰，否则效果不好)
- (3) 然后点击红色×号按钮停止运行程序，此时就可以在右上方的帧缓冲区中看到图像画面，



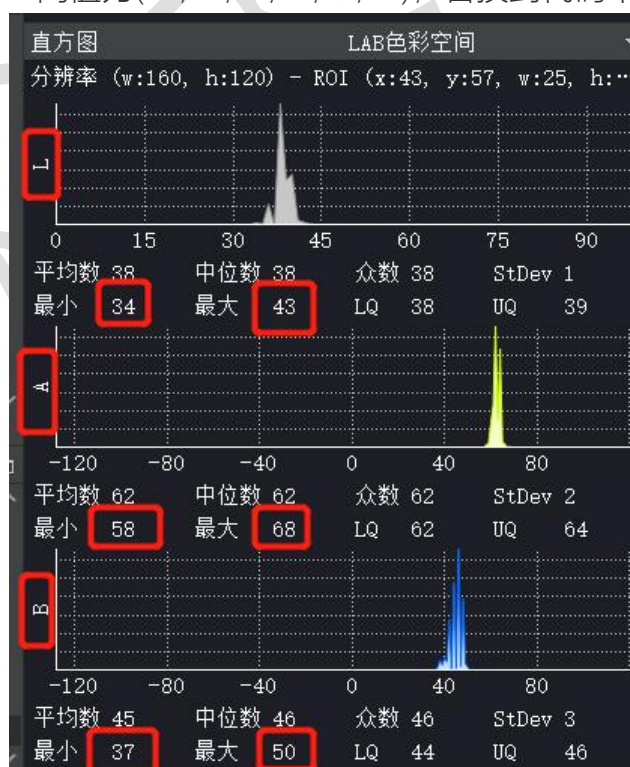
(4) 在'直方图'中选择'LAB 色彩空间',



(5) 在'帧缓冲区'中用鼠标左键将要识别的颜色框出来, 比如这里我将红色木块框出来了



(6) 观察'直方图'中的 L、A、B 的最小值与最大值(也可将鼠标左键分别悬停在 L、A、B 的 x 轴上查看具体数值), 如下图 L 最小 34, 最大 43, A 最小 58, 最大 68, B 最小 37, 最大 50, 则 LAB 阈值为(34,43,58,68,37,50), 替换到代码中使用就可以了。





```
14 import sensor, time
15
16 # 颜色阈值索引, 可以改变索引值来识别不同颜色此处0为红色, 1为
17 threshold_index = 0
18
19 # 设置颜色阈值 此外设置红绿蓝三种颜色的阈值
20 thresholds = (34, 43, 58, 68, 37, 50) # 红色阈值
21             (21, 96, -49, -33, 20, 47), # 绿色阈值
22             (16, 46, 9, 31, -15, -115)] # 蓝色阈值
23
```

3.确定并替换好代码中的颜色阈值之后, 就可以运行此代码了。然后将要识别的物体放在 openmv 摄像头前面随便移动, 看看 ide 中的帧缓冲区是否会将该物体框出来即可。

### 三、运行效果

将要识别的物体放在 openmv 摄像头前面随便移动, ide 中的帧缓冲区里会将识别到的目标颜色的物体框出来。

单颜色识别只会框出要识别的那一种颜色的物体, 而多颜色识别可以将要识别的多种颜色的多个物体同时框出来。

### 四、知识点

关于 `img.find_blobs` 方法可以在官方文档找到

<https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.image.html#>

打开上述网址之后, 按键盘快捷键 `ctrl f`, 然后搜索 `find_blobs`, 可以查找到 `img.find_blobs` 的使用方法及其对象 `blob` 的方法属性的使用。

### 【本课小结】:

本课学习了使用 OpenMV 进行颜色识别, 注意调节颜色阈值

## 第 2 课 颜色跟随

### 【本课目标】：

本课学习使用 openmv 实现云台跟随目标颜色移动的功能

### 【课前准备】：

OpenMV、USB 数据线、扩展板、云台、zlink 模块、总线对接线

### 【本课内容】：

#### 一、代码

```
1.  import sensor, image, time
2.
3.  from pid import PID
4.  from pyb import Servo
5.
6.  target_threshold = (2, 88, -23, 82, 34, 127)  # 此处以黄色为例
7.
8.  # 初始化函数
9.  def init_setup():
10.     global pan_servo, tilt_servo, pan_pid, tilt_pid, sensor, clock
11.     pan_servo = Servo(1)  # 实例化云台靠下的舵机
12.     tilt_servo = Servo(2)  # 实例化云台靠上的舵机
13.
14.     pan_servo.calibration(500,2500,1500)  # 设置舵机脉冲范围为 500-2500，中心位置为 1500
15.     tilt_servo.calibration(500,2500,1500)  # 设置舵机脉冲范围为 500-2500，中心位置为 1500
16.
17.     #pan_pid = PID(p=0.07, i=0, imax=90)  # 脱机运行或者禁用图像传输，使用这个 PID
18.     #tilt_pid = PID(p=0.05, i=0, imax=90)  # 脱机运行或者禁用图像传输，使用这个 PID
19.     pan_pid = PID(p=0.12, i=0, imax=90)  # 在线调试使用这个 PID
20.     tilt_pid = PID(p=0.12, i=0, imax=90)  # 在线调试使用这个 PID
21.
22.     sensor.reset()  # 初始化感光元件
23.     sensor.set_pixformat(sensor.RGB565)  # 图像格式设置为彩色
24.     sensor.set_framesize(sensor.QQVGA)  # 图像大小设置为 QQVGA，即 160x120
25.     sensor.skip_frames(10)  # 跳过 10 帧，使以上设置生效
26.     sensor.set_auto_whitebal(False)  # 关闭白平衡
27.     clock = time.clock()  # 创建时钟对象
28.
29.  # 舵机回归中位
30.  def middle():
31.     pan_servo.angle(0, 1000)  # 舵机在 1 秒内转到 0 度位置
32.     tilt_servo.angle(0, 1000)  # 舵机在 1 秒内转到 0 度位置
33.     time.sleep(1)  # 延时 1 秒
34.
35.  # 查找识别到的目标颜色区域中最大区域的函数
36.  def find_max(blobs):
37.     max_size = 0
38.     for blob in blobs:
39.         if blob[2] * blob[3] > max_size:
```



```

40.         max_blob = blob
41.         max_size = blob[2] * blob[3]
42.     return max_blob
43.
44. # 主函数
45. def main():
46.     while 1:
47.         clock.tick()                # 更新 FPS 帧率时钟
48.         img = sensor.snapshot()      # 拍一张照片并返回图像
49.
50.         blobs = img.find_blobs([target_threshold]) # 查找图像中的目标颜色
51.         if blobs:
52.             max_blob = find_max(blobs)          # 找到最大的目标颜色区域
53.             pan_error = max_blob.cx() - img.width() / 2
54.             tilt_error = max_blob.cy() - img.height() / 2
55.
56.             #print("tilt_error: ", tilt_error)
57.
58.             img.draw_rectangle(max_blob.rect()) # 将最大的目标颜色区域框出来
59.             img.draw_cross(max_blob.cx(), max_blob.cy()) # 在目标颜色区域的中心点处画十字
60.
61.             pan_output = pan_pid.get_pid(pan_error, 1) / 2
62.             tilt_output = tilt_pid.get_pid(tilt_error, 1) / 2
63.             #print("tilt_output", tilt_output)
64.             pan_servo.angle(pan_servo.angle() - pan_output)
65.             tilt_servo.angle(tilt_servo.angle() - tilt_output)
66.
67.             # 异常判断, 定义舵机极限位置
68.             if pan_servo.angle() > 90:
69.                 pan_servo.angle(90)
70.             if pan_servo.angle() < -90:
71.                 pan_servo.angle(-90)
72.             if tilt_servo.angle() > 45:
73.                 tilt_servo.angle(45)
74.             if tilt_servo.angle() < -60:
75.                 tilt_servo.angle(-60)
76.
77. # 程序入口
78. if __name__ == '__main__':
79.     init_setup()    # 执行初始化函数
80.     try:
81.         main()      # 执行主函数
82.     finally:
83.         middle()    # 程序结束时使云台舵机复位

```

## 二、代码使用方法

1. 用 openmv ide 打开 color\_tracking.py, 调节代码中的颜色阈值, 具体调节方法可参考颜色识别那一节。

2. 将 pid.py 文件复制粘贴到 openmv 的盘符中, 然后用 openmv ide 打开

color\_tracking.py 运行使用即可

### 三、运行效果

运行代码后，openmv 摄像头可以识别到红绿蓝三种颜色，当然你也可以继续增加其他颜色的阈值到代码中。

#### 【本课小结】：

本课学习了使用 openmv 实现云台跟随目标颜色移动的功能

## 第 3 课 人脸识别

### 【本课目标】：

本课学习使用 openmv 实现识别人脸的功能

### 【课前准备】：

OpenMV、USB 数据线

### 【本课内容】：

这个例子展示了 OpenMV Cam 的内置人脸检测功能。

人脸检测通过在图像上使用 Haar Cascade 特征检测器来工作。haar 级联是一系列简单的区域对比检查。对于内置的前表面探测器，有 25 个阶段的检查，每个阶段有数百个检查一块。Haar Cascades 运行速度很快，因为只有在以前的阶段过去后才会评估后期阶段。此外，您的 OpenMV 使用称为整体图像的数据结构来在恒定时间内快速执行每个区域对比度检查（特征检测仅为灰度的原因是因为整体图像的空间需求）。

### 一、代码

```
1.  """
2.  人脸识别例程
3.  这个例子展示了 OpenMV Cam 的内置人脸检测功能。
4.  人脸检测通过在图像上使用 Haar Cascade 特征检测器来工作。 haar 级联是一系列简单的区域对比检查。
5.  对于内置的前表面探测器，有 25 个阶段的检查，每个阶段有数百个检查一块。 Haar Cascades 运行速度
6.  很快，因为只有在以前的阶段过去后才会评估后期阶段。 此外，您的 OpenMV 使用称为整体图像的数据
7.  结构来在恒定时间内快速执行每个区域对比度检查
8.  （特征检测仅为灰度的原因是因为整体图像的空间需求）。
9.  """
10. import sensor, image, time
11.
12. # 初始化函数
13. def init_setup():
14.     # 设置为全局变量
15.     global sensor, face_cascade, clock
16.     # 重置感光元件
17.     sensor.reset()
18.     # 感光元件设置
19.     sensor.set_contrast(3)
20.     sensor.set_gainceiling(16)
21.     # HQVGA 和灰度对于人脸识别效果最好，240x160
22.     sensor.set_framesize(sensor.HQVGA)
23.     # 注意人脸识别只能用灰度图哦
24.     sensor.set_pixformat(sensor.GRAYSCALE)
25.     # 创建时钟对象
26.     clock = time.clock()
27.
28. # 加载 Haar 算子
```

```
29.     # 默认情况下, 这将使用所有阶段, 更低的 stages 更快, 但不太准确。
30.     face_cascade = image.HaarCascade("frontalface", stages=25)
31.     # image.HaarCascade(path, stages=Auto)加载一个 haar 模型。haar 模型是二进制文件,
32.     # 这个模型如果是自定义的, 则引号内为模型文件的路径; 也可以使用内置的 haar 模型,
33.     # 比如"frontalface" 人脸模型或者"eye"人眼模型。
34.     # stages 值未传入时使用默认的 stages。stages 值设置的小一些可以加速匹配, 但会降低准确率。
35.     # print(face_cascade)
36.
37.     # 主函数
38.     def main():
39.         clock.tick()                # 更新 FPS 帧率时钟
40.         img = sensor.snapshot()      # 拍摄一张照片
41.
42.         objects = img.find_features(face_cascade, threshold=1, scale=1.25)
43.         # image.find_features(cascade, threshold=0.5, scale=1.5)
44.         # thresholds 越大, 匹配速度越快, 错误率也会上升。scale 可以缩放被匹配特征的大小。
45.         if objects:                  # 检测到人脸时
46.             for r in objects:
47.                 img.draw_rectangle(r) # 在找到的目标上画框, 标记出来
48.                 print(r)             # 打印人脸的左上角坐标、人脸宽度高度
49.             else:                    # 未检测到人脸时
50.                 print("FPS %f" % clock.fps()) # 打印帧率
51.
52.     # 程序入口
53.     if __name__ == '__main__':
54.         init_setup()                # 初始化函数
55.         try:                         # 异常处理
56.             while 1:                 # 无限循环
57.                 main()               # 执行主函数
58.         except:
59.             pass
```

## 二、运行效果

运行之后, 将人脸暴露在 openmv 摄像头画面中时, 它会将人脸框出来。

## 三、知识点

关于 img.find\_features 方法可以在官方文档找到

<https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.image.html#>

打开上述网址之后, 按键盘快捷键 ctrl f, 然后搜索 find\_features, 可以查找到 img.find\_features 的使用方法及 HaarCascade 类 – 特征描述符的使用。

## 【本课小结】：

本课学习了使用 openmv 实现人脸识别

## 第 4 课 人脸跟随

### 【本课目标】：

本课学习使用 openmv 实现云台跟随人脸移动的功能

### 【课前准备】：

OpenMV、USB 数据线、扩展板、云台、zlink 模块、总线对接线

### 【本课内容】：

#### 一、代码

```
1. import sensor, image, time
2.
3. from pid import PID
4. from pyb import Servo
5.
6.
7. # 初始化函数
8. def init_setup():
9.     global pan_servo, tilt_servo, pan_pid, tilt_pid, sensor, clock, face_cascade
10.    pan_servo = Servo(1)                # 实例化云台靠下的舵机
11.    tilt_servo = Servo(2)               # 实例化云台靠上的舵机
12.
13.    pan_servo.calibration(500,2500,1500) # 设置舵机脉冲范围为 500-2500，中心位置为 1500
14.    tilt_servo.calibration(500,2500,1500) # 设置舵机脉冲范围为 500-2500，中心位置为 1500
15.
16.    #pan_pid = PID(p=0.07, i=0, imax=90) # 脱机运行或者禁用图像传输，使用这个 PID
17.    #tilt_pid = PID(p=0.05, i=0, imax=90) # 脱机运行或者禁用图像传输，使用这个 PID
18.    pan_pid = PID(p=0.12, i=0, imax=90)  # 在线调试使用这个 PID
19.    tilt_pid = PID(p=0.12, i=0, imax=90) # 在线调试使用这个 PID
20.
21.    sensor.reset()                     # 初始化感光元件
22.    sensor.set_contrast(3)              # 对比度
23.    sensor.set_gainceiling(16)          # 自动增益
24.    sensor.set_pixformat(sensor.GRAYSCALE) # 图像格式设置为灰度
25.    sensor.set_framesize(sensor.HQVGA)  # 图像大小设置为 HQVGA，即 240x160
26.    sensor.skip_frames(10)              # 跳过 10 帧，使以上设置生效
27.    sensor.set_auto_whitebal(False)      # 关闭白平衡
28.    clock = time.clock()                # 创建时钟对象
29.    face_cascade = image.HaarCascade("frontalface", stages=25)
30.
31. # 舵机回归中位
32. def middle():
33.     pan_servo.angle(0, 1000)           # 舵机在 1 秒内转到 0 度位置
34.     tilt_servo.angle(0, 1000)          # 舵机在 1 秒内转到 0 度位置
35.     time.sleep(1)                     # 延时 1 秒
36.
37. # 查找识别到的目标颜色区域中最大区域的函数
38. def find_max(blobs):
```

```

39.     max_size = 0
40.     for blob in blobs:
41.         if blob[2] * blob[3] > max_size:
42.             max_blob = blob
43.             max_size = blob[2] * blob[3]
44.     return max_blob
45.
46. # 主函数
47. def main():
48.     while 1:
49.         clock.tick()                # 更新 FPS 帧率时钟
50.         img = sensor.snapshot()      # 拍一张照片并返回图像
51.
52.         faces = img.find_features(face_cascade, threshold=0.75, scale=1.25)
53.         if faces:
54.             face = find_max(faces)
55.             cx = int(face[0] + face[2] / 2)
56.             cy = int(face[1] + face[3] / 2)
57.             pan_error = cx - img.width() / 2
58.             tilt_error = cy - img.height() / 2
59.
60.             #print("tilt_error: ", tilt_error)
61.
62.             img.draw_rectangle(face)    # 将最大的目标区域框出来
63.             img.draw_cross(cx, cy)     # 在目标区域的中心点处画十字
64.
65.             pan_output = pan_pid.get_pid(pan_error, 1) / 2
66.             tilt_output = tilt_pid.get_pid(tilt_error, 1) / 2
67.             #print("tilt_output", tilt_output)
68.             pan_servo.angle(pan_servo.angle() - pan_output)
69.             tilt_servo.angle(tilt_servo.angle() - tilt_output)
70.
71.             # 异常判断, 定义舵机极限位置
72.             if pan_servo.angle() > 90:
73.                 pan_servo.angle(90)
74.             if pan_servo.angle() < -90:
75.                 pan_servo.angle(-90)
76.             if tilt_servo.angle() > 45:
77.                 tilt_servo.angle(45)
78.             if tilt_servo.angle() < -60:
79.                 tilt_servo.angle(-60)
80.
81. # 程序入口
82. if __name__ == '__main__':
83.     init_setup()    # 执行初始化函数
84.     try:
85.         main()      # 执行主函数
86.     finally:
87.         middle()    # 程序结束时使云台舵机复位

```

## 二、代码使用方法



先将 pid.py 文件保存到 openmv 的盘符中，然后用 openmv ide 打开 face\_tracking.py 运行使用即可

### 三、运行效果

人脸在 openmv 摄像头的视野中移动时，云台会跟着人脸移动

### 四、知识点

关于 img.find\_features 方法可以在官方文档找到

<https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.image.html#>

打开上述网址之后，按键盘快捷键 ctrl f，然后搜索 find\_features，可以查找到 img.find\_features 的使用方法及 HaarCascade 类 - 特征描述符的使用。

### 【本课小结】：

本课学习了使用 openmv 实现人脸跟随的功能

## 第 5 课 分辨不同人脸

### 【本课目标】：

本课学习使用 openmv 实现人脸面部特征识别的功能，即分辨不同人脸

### 【课前准备】：

OpenMV（需要插上 SD 卡）、USB 数据线

### 【本课内容】：

注意：

1. 本节课学习的是使用 openmv 实现分辨不同人脸的功能，openmv 需要插上 SD 卡来使用，否则内置 flash 可能不够用，因为我们要将一些人脸图片保存到 openmv 中。

2. 本节对应的源码例程中有两个文件：face\_collection.py 和 face\_recognition.py  
face\_collection.py 是采集人脸，face\_recognition.py 是分辨不同人脸。

要先根据教程采集人脸，然后再来分辨人脸

### 一、代码

采集人脸的代码：

```
1.  # 人脸图片采集
2.  # 采集人脸图片到 openmv 中
3.  # 先在 openmv 的盘符中新建文件夹 ZLTech，然后在该文件夹中新建文件夹 s1，s2，要采集几个人就新建到 s 几
4.  # 红灯亮做准备，蓝灯亮是正在拍照
5.  import sensor, image, pyb
6.
7.  RED_LED_PIN = 1      # 红灯引脚
8.  BLUE_LED_PIN = 3     # 蓝灯引脚
9.  SAVE_PATH = 'ZLTech' # 图片保存的位置
10. num = 1 # 设置被拍摄者序号，第一个人的图片保存到 s1 文件夹，第二个人的图片保存到 s2 文件夹，以此类推。每次更换拍摄者时，
    修改 num 值。
11. n = 20 # 设置每个人拍摄图片数量。
12.
13. # 初始化函数
14. def init_setup():
15.     global sensor
16.     sensor.reset()          # 初始化感光元件
17.     sensor.set_pixformat(sensor.GRAYSCALE) # 设置图像格式为灰度图
18.     sensor.set_framesize(sensor.B128X128) # 设置图像大小为 128x128
19.     sensor.set_windowing((92,112))      # 设置窗口大小为 92x112
20.     sensor.skip_frames(10)              # 使以上设置生效
21.     sensor.skip_frames(time=2000)
22.
23. # 主函数
```

```
24. def main():
25.     global n
26.     #连续拍摄 n 张照片，每间隔 3s 拍摄一次。
27.     while (n):
28.         #红灯亮
29.         pyb.LED(RED_LED_PIN).on()
30.         sensor.skip_frames(time=3000) # 等待 3s，准备一下表情。
31.
32.         #红灯灭，蓝灯亮
33.         pyb.LED(RED_LED_PIN).off()
34.         pyb.LED(BLUE_LED_PIN).on()
35.
36.         #保存截取到的图片到 SD 卡
37.         print(n)
38.         sensor.snapshot().save("%s/s%s/%s.pgm" % (SAVE_PATH, num, n) ) # or "example.bmp" (or others)
39.
40.         n -= 1
41.
42.         pyb.LED(BLUE_LED_PIN).off() # 蓝灯灭
43.         print("Done! Reset the camera to see the saved image.")
44.
45. # 程序入口
46. if __name__ == '__main__':
47.     init_setup() # 执行初始化函数
48.     try:         # 异常处理
49.         main()   # 执行主函数
50.     except:
51.         pass
```

### 分辨人脸的代码：

```
1. # 使用 LBP 分辨不同人脸
2. # 运行此代码前请先运行采集人脸的代码来采集人脸，如果已采集完成，可以运行此代码
3.
4. import sensor, time, image, pyb
5.
6. NUM_SUBJECTS = 2 # 图像库中不同人数
7. NUM_SUBJECTS_IMGS = 20 # 每人有 20 张样本图片
8. name_list = [None, '迪丽热巴', '鞠婧祎'] # 对应人的名字
9.
10. # 初始化函数
11. def init_setup():
12.     global sensor, face_cascade # 设置为全局变量
13.     sensor.reset() # 初始化感光元件
14.     sensor.set_pixformat(sensor.GRAYSCALE) # 将图像格式设置为灰度
15.     sensor.set_framesize(sensor.B128X128) # 将图像大小设置为 128x128
16.     face_cascade = image.HaarCascade("frontalface", stages=25)
17.     sensor.set_windowing((92,112)) # 将窗口大小设置为 92x112
18.     sensor.skip_frames(10)
19.     sensor.skip_frames(time=1000) # 等待 1s，使以上设置生效
20.
21. def min(pmin, a, s):
22.     global num
```

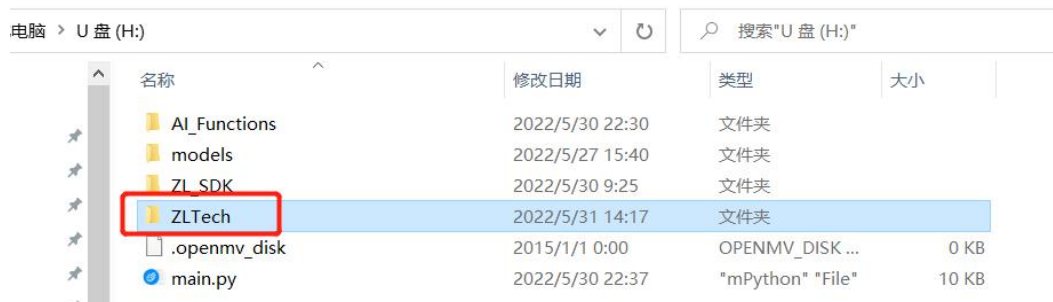
```

23.     if a < pmin:
24.         pmin = a
25.         num = s
26.     return pmin
27.
28. # 主函数
29. def main():
30.     global num
31.     # 拍摄当前人脸
32.     img = sensor.snapshot()
33.     # d0 为当前人脸的 lbp 特征
34.     d0 = img.find_lbp((0, 0, img.width(), img.height()))
35.     objects = img.find_features(face_cascade, threshold=1, scale=1.25)
36.     if objects:                                # 检测到人脸时
37.         img = None
38.         pmin = 999999
39.         num = 0
40.         for s in range(1, NUM_SUBJECTS+1):
41.             dist = 0
42.             for i in range(1, NUM_SUBJECTS_IMGS+1):
43.                 img = image.Image("ZLTech/s%d/%d.pgm"%(s, i))
44.                 d1 = img.find_lbp((0, 0, img.width(), img.height()))
45.                 # d1 为第 s 文件夹中的第 i 张图片的 lbp 特征
46.                 dist += image.match_descriptor(d0, d1) # 计算 d0 d1 即样本图像与被检测人脸的特征差异度。
47.                 print("Average dist for subject %d: %d"%(s, dist/NUM_SUBJECTS_IMGS)) # 打印与每个人的特征差异度
48.                 pmin = min(pmin, dist/NUM_SUBJECTS_IMGS, s) # 特征差异度越小, 被检测人脸与此样本更相似更匹配。
49.
50.             print(name_list[num]) # num 为当前最匹配的人的编号, 然后从名字列表中取出对应人的名字。
51.
52. # 程序入口
53. if __name__ == '__main__':
54.     init_setup() # 执行初始化函数
55.     try:         # 异常处理
56.         while 1:
57.             main() # 执行主函数
58.     except:
59.         pass

```

## 二、代码使用方法

1. 在 openmv 的盘符中新建文件夹 ZLTech, 然后在该文件夹中新建文件夹 s1, s2..., 要采集几个人就新建到 s 几, 比如我要分辨两个人, 我就在 ZLTech 文件夹中新建 s1 和 s2





2.在 openmv ide 中打开采集人脸的代码 face\_collection.py

```
0
7 import sensor, image, pyb
8
9 RED_LED_PIN = 1          # 红灯引脚
10 BLUE_LED_PIN = 3         # 蓝灯引脚
11 SAVE_PATH = 'ZLTech'    # 图片保存的位置
12 num = 1 # 设置被拍摄者序号，第一个人的图片保存到s1文件夹，第二个人的图片保
13
14 n = 20 # 设置每个人拍摄图片数量。
15
```

运行程序即可开始拍照。num=1 表示拍第一个人的脸，当你拍完第一个人的 20 张照片后，可以将代码中的 num=1 改为 num=2，然后再次运行程序来拍第二个人的脸。

拍照时，红灯亮为做准备，蓝灯亮表示正在拍照。

3.在 openmv ide 中打开分辨人脸的代码 face\_recognition.py

```
# 使用LBP分辨不同人脸
# 运行此代码前请先运行采集人脸的代码来采集人脸，如果已采集完成，可以运行此代码

import sensor, time, image, pyb

NUM_SUBJECTS = 2          # 图像库中不同人数
NUM_SUBJECTS_IMGS = 20    # 每人有20张样本图片
name_list = [None, '迪丽热巴', '鞠婧祎'] # 对应人的名字
```

NUM\_SUBJECTS = 2 表示采集人脸时一共采集了两个人的照片，根据实际情况修改即可，我这里是两个人，所以这里 NUM\_SUBJECTS = 2

name\_list = [None, '迪丽热巴', '鞠婧祎']是对应人的名字，比如我的 s1 中存的照片对应的是'迪丽热巴'，s2 中存的照片对应的是'鞠婧祎'，你可以根据你的实际情况进行增加修改等。

4.运行分辨人脸的代码 face\_recognition.py 即可

### 三、运行效果

运行代码后，将已采集过图片的那个人的脸暴露在摄像头画面中时，ide 的串行端口窗口中就会打印出这个人的名字。我们有提供的测试图片。

### 四、知识点

关于 `img.find_lbp` 方法可以在官方文档找到

<https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.image.html#>

打开上述网址之后，按键盘快捷键 `ctrl f`，然后搜索 `find_lbp`，可以查找到 `img.find_lbp` 的使用方法。

### 【本课小结】：

本课学习了使用 openmv 通过 LBP 分辨不同人脸



## 第 6 课 线段识别

### 【本课目标】：

本课学习使用 openmv 实现识别线段的功能

### 【课前准备】：

OpenMV、USB 数据线

### 【本课内容】：

#### 一、代码

```
1.  # 线段检测例程
2.  #
3.  # 在图像中查找线段。对于在图像中找到的每个线对象，都会返回一个包含线条旋转的线对象。
4.
5.  # find_line_segments()找到有限长度的线（但是很慢）。
6.  # 用 find_line_segments()找到非无限的线（而且速度很快）。
7.
8.  enable_lens_corr = False          # 畸变矫正，打开可以获得更直的线条...
9.
10. import sensor, image, time
11.
12. # 初始化函数
13. def init_setup():
14.     global sensor, clock           # 设置为全局变量
15.     sensor.reset()                 # 重置感光元件
16.     sensor.set_pixformat(sensor.RGB565) # 此处设置的是彩色，如果是灰度会更快
17.     sensor.set_framesize(sensor.QQVGA) # 设置图像分辨率为 QQVGA，即 160x120
18.     sensor.skip_frames(time=2000)    # 等待 2 秒，使以上设置生效
19.     clock = time.clock()            # 创建时钟对象
20.
21. # 所有线段都有 `x1()`, `y1()`, `x2()`, 和 `y2()` 方法来获得他们的终点
22. # 一个 `line()` 方法来获得所有上述的四个元组值，可用于 `draw_line()`.
23.
24. # 主函数
25. def main():
26.     while 1:
27.         clock.tick()                # 更新 FPS 帧率时钟
28.         img = sensor.snapshot()      # 拍一张图片并返回图像
29.         if enable_lens_corr:
30.             img.lens_corr(1.8)       # 畸变矫正，for 2.8mm lens...
31.
32.         # `merge_distance` 控制附近行的合并。 在 0（默认），没有合并。
33.         # 在 1 处，任何距离另一条线一个像素点的线都被合并...等等，
34.         # 因为你增加了这个值。 您可能希望合并线段，因为线段检测会产生大量的线段结果。
35.
36.         # `max_theta_diff` 控制要合并的任何两线段之间的最大旋转差异量。
37.         # 默认设置允许 15 度。
38.
```



```
39.         for l in img.find_line_segments(merge_distance=0, max_theta_diff=5):
40.             img.draw_line(l.line(), color=(255, 0, 0))
41.             print(l)
42.
43.         print("FPS %f" % clock.fps())    # 打印帧率, 注: 实际 FPS 更高, 流 FB 使它更慢。
44.
45.     # 程序入口
46.     if __name__ == '__main__':
47.         init_setup() # 执行初始化函数
48.         try:
49.             main()    # 执行主函数
50.         except:
51.             pass
```

## 二、运行效果

运行代码后, openmv 识别到的线段都会被标出来

## 三、知识点

关于 `img.find_line_segments` 方法可以在官方文档找到

<https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.image.html#>

打开上述网址之后, 按键盘快捷键 `ctrl f`, 然后搜索 `find_line_segments`, 可以查找  
到 `image.find_line_segments` 的使用方法及 `Line` 类 - 直线对象的使用。

### 【本课小结】：

本课学习了使用 openmv 识别线段。

## 第 7 课 直线识别

### 【本课目标】：

本课学习使用 openmv 实现识别直线的功能

### 【课前准备】：

OpenMV、USB 数据线

### 【本课内容】：

本课学习如何在图像中查找线条。对于在图像中找到的每个线对象，都会返回一个包含线条旋转的线对象。

### 一、代码

```
1.  # 识别直线例程
2.  #
3.  # 这个例子展示了如何在图像中查找线条。对于在图像中找到的每个线对象，都会返回一个包含线条旋转的线对象。
4.
5.  # 注意：线条检测是通过使用霍夫变换完成的：
6.  # http://en.wikipedia.org/wiki/Hough\_transform
7.  # 请阅读以上关于“theta”和“rho”的更多信息。
8.
9.  # find_lines () 找到无限长度的线。使用 find_line_segments () 来查找非无限线。
10.
11. enable_lens_corr = False                # 畸变矫正，打开可以获得更直的线条...
12. import sensor, image, time
13.
14. # 初始化函数
15. def init_setup():
16.     global sensor, clock, min_degree, max_degree # 设置为全局变量
17.     sensor.reset()                               # 重置感光元件
18.     sensor.set_pixformat(sensor.RGB565)          # 此处设置的是彩色，如果是灰度会更快
19.     sensor.set_framesize(sensor.QQVGA)           # 设置图像分辨率为 QQVGA，即 160x120
20.     sensor.skip_frames(time=2000)                # 等待 2 秒，使以上设置生效
21.     clock = time.clock()                         # 创建时钟对象
22.
23.     # 所有的线对象都有一个`theta ()`方法来获取它们的旋转角度。您可以根据旋转角度来过滤线条。
24.     min_degree = 0
25.     max_degree = 179
26.
27.     # 所有线段都有`x1()`，`y1()`，`x2()`，and `y2()`方法来获得他们的终点
28.     # 一个`line()`方法来获得所有上述的四个元组值，可用于`draw_line()`。
29.
30. # 主函数
31. def main():
32.     while 1:
33.         clock.tick()                             # 更新 FPS 帧率时钟
34.         img = sensor.snapshot()                   # 拍一张图片并返回图像
35.         if enable_lens_corr:
36.             img.lens_corr(1.8)                    # for 2.8mm lens...
```

```

37.
38.     # `threshold` 控制从霍夫变换中监测到的直线。只返回大于或等于阈值的直线。应用程序的阈值正确值取决于图像。
39.     # 注意：一条直线的大小是组成直线所有索贝尔滤波像素大小的总和。
40.
41.     # `theta_margin` 和 `rho_margin` 控件合并相似的直线。如果两直线的 theta 和 p 值差异小于边际，则它们合并。
42.
43.     for l in img.find_lines(threshold=1000, theta_margin=25, rho_margin=25):
44.         if (min_degree <= l.theta()) and (l.theta() <= max_degree):
45.             img.draw_line(l.line(), color=(255, 0, 0))
46.             print(l)
47.
48.     print("FPS %f" % clock.fps())          # 打印帧率，注：实际 FPS 更高，流 FB 使它更慢。
49.
50. # 程序入口
51. if __name__ == '__main__':
52.     init_setup() # 执行初始化函数
53.     try:
54.         main()   # 执行主函数
55.     except:
56.         pass
57.
58. # 关于负 rho 值：
59. #
60. # A [theta+0:-rho] tuple is the same as [theta+180:+rho].
61. # A [theta+0:-rho] 元组与 [theta+180:+rho] 相同。

```

## 二、运行效果

运行代码后，openmv 会将识别到的直线都标出来。

注意：此处的直线包括一些线只要它有一端被隔断在画面之外也算是直线。

## 三、知识点

关于 `img.find_lines` 方法可以在官方文档找到

<https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.image.html#>

打开上述网址之后，按键盘快捷键 `ctrl f`，然后搜索 `find_lines`，可以查找到 `image.find_lines` 的使用方法及 `Line` 类 – 直线对象的使用。

## 【本课小结】：

本课学习了使用 openmv 识别直线。

## 第 8 课 圆形识别

### 【本课目标】：

本课学习使用 openmv 实现识别圆形的功能。

### 【课前准备】：

OpenMV、USB 数据线

### 【本课内容】：

本节学习使用霍夫变换在图像中查找圆

#### 一、代码

```
1.  # 圆形检测例程
2.  #
3.  # 用霍夫变换在图像中找到圆。
4.  # https://en.wikipedia.org/wiki/Circle\_Hough\_Transform
5.  #
6.  # 请注意，find_circles（）方法将只能找到完全在图像内部的圆。圈子之外的图像/ roi 被忽略...
7.
8.  import sensor, image, time
9.
10. # 初始化函数
11. def init_setup():
12.     global sensor, clock          # 设置为全局变量
13.     sensor.reset()                # 初始化感光元件
14.     sensor.set_pixformat(sensor.RGB565) # 将图像格式设置为彩色
15.     sensor.set_framesize(sensor.QQVGA) # 将图像大小设置为 QQVGA (160x120)
16.     sensor.skip_frames(10)         # 跳过一些帧，使以上设置生效。
17.     clock = time.clock()           # 创建时钟对象
18.
19. # 主函数
20. def main():
21.     # 更新 FPS 帧率时钟
22.     clock.tick()
23.
24.     # # 拍一张照片并返回图像，lens_corr(1.8)为畸变矫正
25.     img = sensor.snapshot().lens_corr(1.8)
26.
27.     # Circle 对象有四个值：x, y, r (半径), 和 magnitude。
28.     # magnitude 是检测圆的强度。越高越好
29.
30.     # roi 是一个用以复制的矩形的感兴趣区域(x, y, w, h)。如果未指定，
31.     # ROI 即图像矩形。操作范围仅限于 roi 区域内的像素。
32.
33.     # x_stride 是霍夫变换时需要跳过的 x 像素的数量。若已知圆较大，可增加 x_stride 。
34.
35.     # y_stride 是霍夫变换时需要跳过的 y 像素的数量。若已知直线较大，可增加 y_stride 。
36.
37.     # threshold 控制从霍夫变换中检测到的圆。只返回大于或等于阈值的圆。
```

```
38.     # 应用程序的阈值正确值取决于图像。注意：一个圆的大小是组成圆所有索贝尔滤波像素大小的总和。
39.
40.     # x_margin 控制所检测的圆的合并。圆像素为 x_margin 、 y_margin 和 r_margin 的部分合并。
41.
42.     # y_margin 控制所检测的圆的合并。圆像素为 x_margin 、 y_margin 和 r_margin 的部分合并。
43.
44.     # r_margin 控制所检测的圆的合并。圆像素为 x_margin 、 y_margin 和 r_margin 的部分合并。
45.
46.     # r_min, r_max 和 r_step 控制测试圆的半径。缩小测试圆半径的数量可以大大提升性能。
47.     # threshold=3500 比较合适。如果视野中检测到的圆过多，请增大阈值；相反，如果视野中检测到的圆过少，请减少阈值。
48.     for c in img.find_circles(threshold=3500, x_margin=10, y_margin=10, r_margin=10, r_min=2, r_max=100, r_step=2):
49.         img.draw_circle(c.x(), c.y(), c.r(), color = (255, 0, 0))
50.         print(c)
51.
52.     print("FPS %f" % clock.fps()) # 打印帧率
53.
54. # 程序入口
55. if __name__ == '__main__':
56.     init_setup() # 执行初始化函数
57.     try:
58.         while 1:
59.             main() # 执行主函数
60.     except:
61.         pass
```

## 二、运行效果

运行代码后，openmv 会将识别到的圆都用红线画出来。

注意：黑色实心圆会更易识别。

## 三、知识点

关于 img.find\_circles 方法可以在官方文档找到

<https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.image.html#>

打开上述网址之后，按键盘快捷键 ctrl f，然后搜索 find\_circles，可以查找到 image.find\_circles 的使用方法及 Circle 类 – 圆形对象的使用。

### 【本课小结】：

本课学习了使用 openmv 识别圆形。

注意黑色实心圆会更易识别。

## 第 9 课 矩形识别

### 【本课目标】：

本课学习使用 openmv 实现识别矩形的功能

### 【课前准备】：

OpenMV、USB 数据线

### 【本课内容】：

本节学习使用四元检测算法来检测矩形。

### 一、代码

```
1.  # 矩形识别例题
2.  #
3.  # 使用 april 标签代码中的四元检测代码在图像中找到矩形。
4.  # 四元检测算法以非常稳健的方式检测矩形，并且比基于霍夫变换的方法好得多。
5.  # 例如，即使镜头失真导致这些矩形看起来弯曲，它仍然可以检测到矩形。圆角矩形是没有问题的！
6.  # (但是，这个代码也会检测小半径的圆)...
7.
8.  import sensor, image, time
9.
10. # 初始化函数
11. def init_setup():
12.     global sensor, clock          # 设置为全局变量
13.     sensor.reset()                # 重置感光元件
14.     sensor.set_pixformat(sensor.RGB565) # 将图像格式设置为彩色
15.     sensor.set_framesize(sensor.QQVGA) # 设置图像分辨率为 QQVGA，即 160x120
16.     sensor.skip_frames(10)         # 跳过 10 帧，使以上设置生效
17.     clock = time.clock()           # 创建时钟对象
18.
19. # 主函数
20. def main():
21.     clock.tick()                  # 更新 FPS 帧率时钟
22.     img = sensor.snapshot()        # 拍一张图片并返回图像
23.
24.     # 下面的`threshold`应设置为足够高的值，以滤除在图像中检测到的具有低边缘幅度的噪声矩形。最适用与背景形成鲜明对比的
        矩形。
25.     for r in img.find_rects(threshold=15000):
26.         img.draw_rectangle(r.rect(), color=(255, 0, 0)) # 用红线框出矩形
27.         for p in r.corners():
28.             img.draw_circle(p[0], p[1], 5, color=(0, 255, 0)) # 在矩形端点处画绿色圆
29.         print(r)
30.
31.     print("FPS %f" % clock.fps()) # 打印帧率，注：实际 FPS 更高，流 FB 使它更慢。
32.
33. # 程序入口
34. if __name__ == '__main__':
35.     init_setup() # 执行初始化函数
36.     try:
```

```
37.         while 1:
38.             main()    # 执行主函数
39.         except:
40.             pass
```

## 二、运行效果

代码运行后，openmv 会将识别到的矩形框出来，并且四个顶点也会用小圆圈框出来。

注意：此代码也会将小半径的圆识别成矩形。

黑色实心矩形会更易识别。

## 三、知识点

关于 `img.find_rects` 方法可以在官方文档找到

<https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.image.html#>

打开上述网址之后，按键盘快捷键 `ctrl f`，然后搜索 `find_rects`，可以查找到 `image.find_rects` 的使用方法及 `Rect` 类 - 矩形对象的使用。

### 【本课小结】：

本课学习了使用 openmv 识别矩形。

注意：有些小半径的圆形也会被识别成矩形。

## 第 10 课 AprilTag 标签识别

### 【本课目标】：

本课学习使用 openmv 实现识别 AprilTag 标签信息的功能

### 【课前准备】：

OpenMV、USB 数据线

### 【本课内容】：

#### 一、代码

# AprilTag 标签识别示例

```
import sensor, time
```

# 初始化函数

```
def init_setup():
```

```
    global sensor, clock
```

```
    sensor.reset()
```

```
    sensor.set_pixformat(sensor.RGB565)
```

```
    sensor.set_framesize(sensor.QQVGA)
```

QQVGA, 即 160x120

```
    sensor.skip_frames(time=2000)
```

```
    sensor.set_auto_gain(False)
```

```
prevent image washout...
```

```
    sensor.set_auto_whitebal(False)
```

```
prevent image washout...
```

```
    clock = time.clock()
```

# 设置为全局变量

# 重置感光元件

# 此处设置的是彩色

# 设置图像分辨率为

# must turn this off to

# must turn this off to

# 创建时钟对象

# 主函数

```
def main():
```

```
    clock.tick()
```

```
    img = sensor.snapshot()
```

```
    apriltags = img.find_apriltags()
```

```
    for tag in apriltags:
```

```
        img.draw_rectangle(tag.rect, color=(255, 0, 0))    # 用红线将
```

AprilTag 框出来

```
        img.draw_cross(tag.cx, tag.cy, color=(0, 255, 0))
```

```
        #print(tag)
```

# 打印 AprilTag 的所有信息

```
        apriltag_id = tag.id
```

# 获取 AprilTag 的 id

```
        apriltag_family = tag.name
```

# 获取 AprilTag 所属家族

```
        print('AprilTag 的 id 为 %s, 所属家族为 %s' % (apriltag_id,
```

```
        apriltag_family))    # 打印 AprilTag 的 id 及所属家族
```



```
#if not apriltags:
    时打印帧率，注：实际 FPS 更高，流 FB 使它更慢。
    #print("FPS %f" % clock.fps())

# 程序入口
if __name__ == '__main__':
    init_setup()
    try:
        while 1:
            main()
    except:
        pass

# 如果图像中未发现 AprilTag
# 执行初始化函数
# 执行主函数
```

## 二、运行效果

代码运行后，openmv 会将识别到的 AprilTag 标签的 id 和所属家族打印到串行端口窗口中。

## 三、知识点

关于 `img.find_apriltags` 方法可以在官方文档找到

<https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.image.html#>

打开上述网址之后，按键盘快捷键 `ctrl f`，然后搜索 `find_apriltags`，可以查找到 `image.find_apriltags` 的使用方法及 `AprilTag` 类 – `AprilTag` 对象的使用。

### 【本课小结】：

本课学习了使用 openmv 实现识别 AprilTag 标签信息的功能。

## 第 11 课 二维码识别

### 【本课目标】：

本课学习使用 openmv 实现识别二维码信息的功能。

### 【课前准备】：

OpenMV、USB 数据线

### 【本课内容】：

本节学习 OpenMV Cam 使用镜头校正来检测 QR 码的功能。

#### 一、代码

```
1. # 二维码识别示例
2. #
3. # 这个例子展示了 OpenMV Cam 使用镜头校正来检测 QR 码的功能（请参阅 qrcodes_with_lens_corr.py 脚本以获得更高的性能）。
4. import sensor, image, time
5.
6. # 初始化函数
7. def init_setup():
8.     global sensor, clock # 设置为全局变量
9.     sensor.reset() # 重置感光元件
10.    sensor.set_pixformat(sensor.RGB565) # 此处设置的是彩色
11.    sensor.set_framesize(sensor.QVGA) # 设置图像分辨率为 QVGA，即 320x240
12.    sensor.skip_frames(10) # 等待一些帧，使以上设置生效
13.    sensor.set_auto_gain(False) # 必须关闭白平衡功能，以防止图像冲洗...
14.    clock = time.clock() # 创建时钟对象
15.
16. # 主函数
17. def main():
18.     clock.tick() # 更新 FPS 帧率时钟
19.     img = sensor.snapshot() # 拍一张图片并返回图像
20.     img.lens_corr(1.8) # 畸变矫正，1.8 的强度参数对于 2.8mm 镜头来说是不错的。
21.     codes = img.find_qrcodes() # 查找图像中的二维码
22.     for code in codes:
23.         img.draw_rectangle(code.rect(), color = (255, 0, 0)) # 用红线将二维码框出来
24.         #print(code) # 打印二维码的所有信息
25.         msg = code.payload() # 获取二维码的内容
26.         print('二维码的内容为: %s' % msg) # 打印二维码的内容
27.     if not codes: # 如果图像中未发现二维码时打印帧率，注：实际 FPS 更高，流 FB 使它更慢。
28.         print("FPS %f" % clock.fps())
29.
30. # 程序入口
31. if __name__ == '__main__':
32.     init_setup() # 执行初始化函数
33.     try:
34.         while 1:
35.             main() # 执行主函数
36.     except:
```

37. pass

## 二、运行效果

代码运行后，openmv 会将识别到的二维码信息打印到串行端口窗口中。

## 三、知识点

关于 `img.find_qrcodes` 方法可以在官方文档找到

<https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.image.html#>

打开上述网址之后，按键盘快捷键 `ctrl f`，然后搜索 `find_qrcodes`，可以查找到 `image.find_qrcodes` 的使用方法及 `QRCode` 类 – 二维码对象的使用。

### 【本课小结】：

本课学习了使用 openmv 实现识别二维码信息的功能

## 第 12 课 条形码识别

### 【本课目标】：

本课学习使用 openmv 实现识别条形码信息的功能

### 【课前准备】：

OpenMV、USB 数据线

### 【本课内容】：

本节学习使用 OpenMV 来检测条形码。

注意条形码检测不适用于 M4 相机。

条形码检测可以在 OpenMV Cam 的 OV7725 相机模块的 640x480 分辨率下运行。也可在 RGB565 模式下工作，但分辨率较低。也就是说，条形码检测需要更高的分辨率才能正常工作，因此应始终以 640x480 的灰度运行。

### 一、代码

```
1. # 条形码识别例程
2. #
3. # 使用 OpenMV 来检测条形码。条形码检测不适用于 M4 相机。
4.
5. import sensor, image, time, math
6.
7. # 初始化函数
8. def init_setup():
9.     global sensor, clock          # 设置为全局变量
10.    sensor.reset()                 # 重置感光元件
11.    sensor.set_pixformat(sensor.GRAYSCALE) # 此处设置的是灰度
12.    sensor.set_framesize(sensor.VGA)    # 设置图像分辨率为 VGA，即 640x480
13.    sensor.set_windowing((640, 80))    # V Res of 80 == less work (40 for 2X the speed).
14.    sensor.skip_frames(time=2000)      # 等待 2 秒，使以上设置生效
15.    sensor.set_auto_gain(False)        # 必须关闭此功能，以防止图像冲洗...
16.    sensor.set_auto_whitebal(False)    # 必须关闭此功能，以防止图像冲洗...
17.    clock = time.clock()              # 创建时钟对象
18.
19.    # 条形码检测可以在 OpenMV Cam 的 OV7725 相机模块的 640x480 分辨率下运行。
20.    # 条码检测也可在 RGB565 模式下工作，但分辨率较低。
21.    # 也就是说，条形码检测需要更高的分辨率才能正常工作，因此应始终以 640x480 的灰度运行。
22.
23. # 判断条形码类型的函数
24. def barcode_name(code):
25.     if(code.type() == image.EAN2):
26.         return "EAN2"
27.     if(code.type() == image.EAN5):
28.         return "EAN5"
29.     if(code.type() == image.EAN8):
30.         return "EAN8"
```

```

31.     if(code.type() == image.UPCE):
32.         return "UPCE"
33.     if(code.type() == image.ISBN10):
34.         return "ISBN10"
35.     if(code.type() == image.UPCA):
36.         return "UPCA"
37.     if(code.type() == image.EAN13):
38.         return "EAN13"
39.     if(code.type() == image.ISBN13):
40.         return "ISBN13"
41.     if(code.type() == image.I25):
42.         return "I25"
43.     if(code.type() == image.DATABAR):
44.         return "DATABAR"
45.     if(code.type() == image.DATABAR_EXP):
46.         return "DATABAR_EXP"
47.     if(code.type() == image.CODABAR):
48.         return "CODABAR"
49.     if(code.type() == image.CODE39):
50.         return "CODE39"
51.     if(code.type() == image.PDF417):
52.         return "PDF417"
53.     if(code.type() == image.CODE93):
54.         return "CODE93"
55.     if(code.type() == image.CODE128):
56.         return "CODE128"
57.
58. # 主函数
59. def main():
60.     while 1:
61.         clock.tick()                # 更新 FPS 帧率时钟
62.         img = sensor.snapshot()      # 拍一张图片并返回图像
63.         codes = img.find_barcodes() # 从图像中查找条形码
64.         for code in codes:
65.             img.draw_rectangle(code.rect())
66.             print_args = (barcode_name(code), code.payload(), (180 * code.rotation())/math.pi, code.quality())
67.             print("Barcode %s, Payload \"%s\", rotation %f (degrees), quality %d, FPS %f" % print_args)
68.             msg = code.payload()     # 条形码的信息
69.             print('条形码信息为: %s' % msg) # 打印条形码的内容
70.             if not codes:            # 如果图像中未发现条形码时打印帧率, 注: 实际 FPS 更高, 流 FB 使它更慢。
71.                 print("FPS %f" % clock.fps())
72.
73. # 程序入口
74. if __name__ == '__main__':
75.     init_setup()                    # 执行初始化函数
76.     try:
77.         main()                      # 执行主函数
78.     except:
79.         pass

```

## 二、运行效果

代码运行后，openmv 会将识别到的条形码的信息打印到串行端口窗口中。

### 三、知识点

关于 `img.find_barcodes` 方法可以在官方文档找到

<https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.image.html#>

打开上述网址之后，按键盘快捷键 `ctrl f`，然后搜索 `find_barcodes`，可以查找到 `image.find_barcodes` 的使用方法及 `BarCode` 类 – 条形码对象的使用。

### 【本课小结】：

本课学习了使用 openmv 实现识别条形码信息的功能。

## 第 13 课 数字识别（只适用于 openmv4 h7 plus）

### 【本课目标】：

本节学习通过训练神经网络模型实现识别数字的功能

### 【课前准备】：

OpenMV4 H7 PLUS、USB 数据线

### 【本课内容】：

注意：

1. 使用训练神经网络模型的方法实现数字识别只适用于 openmv h7 plus。
2. 在我们的资料中的对应源码例程中有提供的训练好的模型文件 trained.tflite 和配套文件 labels.txt，你可以直接将模型文件 trained.tflite 和配套文件 labels.txt 复制粘贴到 openmv 的盘符中，然后在 ide 中运行代码 num\_detection.py 来使用即可，可以用提供的配套的测试图片来进行测试。
3. 你也可以自己训练想要识别的一些数字的神经网络模型来使用，具体训练方法会在下面讲到。

### 一、代码

```
1.  # TensorFlow 数字识别
2.
3.  import sensor, image, time, os, tf, uos, gc
4.
5.  net = None
6.  labels = None
7.
8.  # 初始化函数
9.  def num_init_setup():
10.     global sensor, net, labels, clock    # 设置为全局变量
11.     sensor.reset()                        # 初始化感光元件
12.     sensor.set_pixformat(sensor.GRAYSCALE) # 设置图像格式为灰度
13.     sensor.set_framesize(sensor.QVGA)     # 设置图像大小为 QVGA (320x240)
14.     sensor.set_windowing((240, 240))      # 设置窗口大小为 240x240
15.     sensor.skip_frames(10)                # 跳过一些帧，使以上设置生效
16.
17.     try:
18.         # 加载模型文件
19.         net = tf.load("trained.tflite", load_to_fb=uos.stat('trained.tflite')[6] > (gc.mem_free() - (64*1024)))
20.     except Exception as e:
21.         print(e)
22.         raise Exception('Failed to load "trained.tflite", did you copy the .tflite and labels.txt file onto the mass-storage device? (' + str(e) + ')')
23.
```

```
24.     try:
25.         labels = [line.rstrip('\n') for line in open("labels.txt")]
26.     except Exception as e:
27.         raise Exception('Failed to load "labels.txt", did you copy the .tflite and labels.txt file onto the
           mass-storage device? (' + str(e) + ')')
28.
29.     clock = time.clock()          # 创建时钟对象
30.
31. # 主函数
32. def num_main():
33.     while 1:
34.         clock.tick()              # 更新 FPS 帧率时钟
35.         img = sensor.snapshot()   # 拍一张照片并返回图像
36.
37.         # default settings just do one detection... change them to search the image...
38.         for obj in net.classify(img, min_scale=1.0, scale_mul=0.8, x_overlap=0.5, y_overlap=0.5):
39.             #print("*****\nPredictions at [x=%d,y=%d,w=%d,h=%d]" % obj.rect())
40.             img.draw_rectangle(obj.rect())
41.             # 将分类和对应的相似度以列表套元组形式返回
42.             predictions_list = list(zip(labels, obj.output()))
43.             # 打印分类和与图像中检测到的物体的对应的相似度
44.             #for i in range(len(predictions_list)):
45.                 #print("%s = %f" % (predictions_list[i][0], predictions_list[i][1]))
46.             if max(obj.output()) > 0.7: # 当识别到的最大的相似度大于 0.7 时才认为是识别到了数字
47.                 # 将识别到的数字赋值给 num
48.                 num = labels[obj.output().index(max(obj.output()))]
49.                 # 打印识别到的数字
50.                 print('识别到的数字是 %s' % num)
51.
52.             #print(clock.fps(), "fps") # 打印帧率
53.
54. # 程序入口
55. if __name__ == '__main__':
56.     num_init_setup() # 执行初始化函数
57.     try:
58.         num_main()   # 执行主函数
59.     except:
60.         pass
```

## 二、代码使用方法

将 labels.txt 和 trained.tflite 复制粘贴到 openmv 的盘符中，然后在 openmv ide 中打开 num\_detection.py 运行使用即可，可以使用提供的测试图片来进行测试。

## 三、运行效果

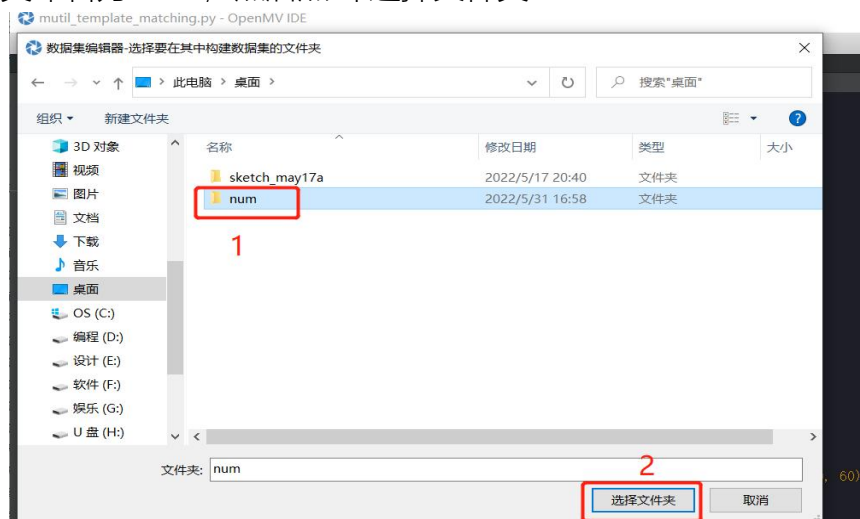
代码运行后，openmv 会将识别到的数字在串行终端窗口打印出来。

## 四、知识点

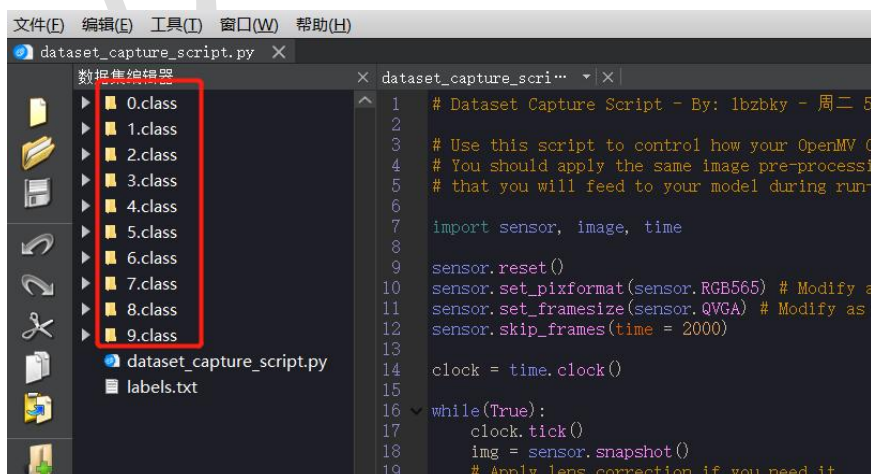


## 训练神经网络模型的方法：

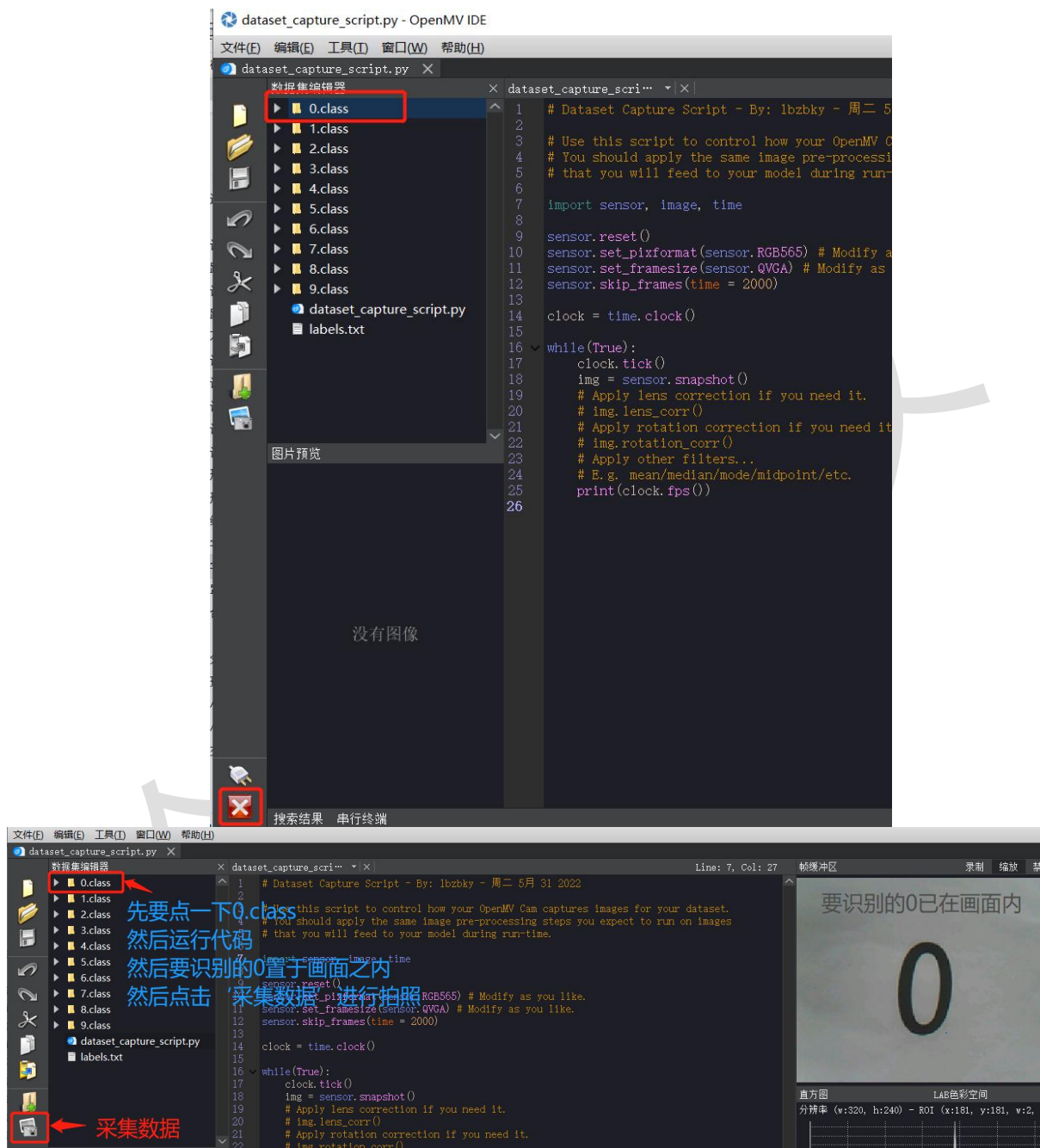
1. 打开 openmv ide, 点击'工具'-->'数据集编辑器', 选择'新数据集', 然后点击鼠标右键, 新建一个文件夹命名为 num, 然后点击'选择文件夹'



2. 在出现的界面中, 点击'新类文件夹'图标来创建子文件夹, 然后为其命名 (以分类命名, 比如要识别的是 0, 就以 0 命名), 共创建 0 到 9 共 10 个子文件夹。都创建完成之后如下所示:



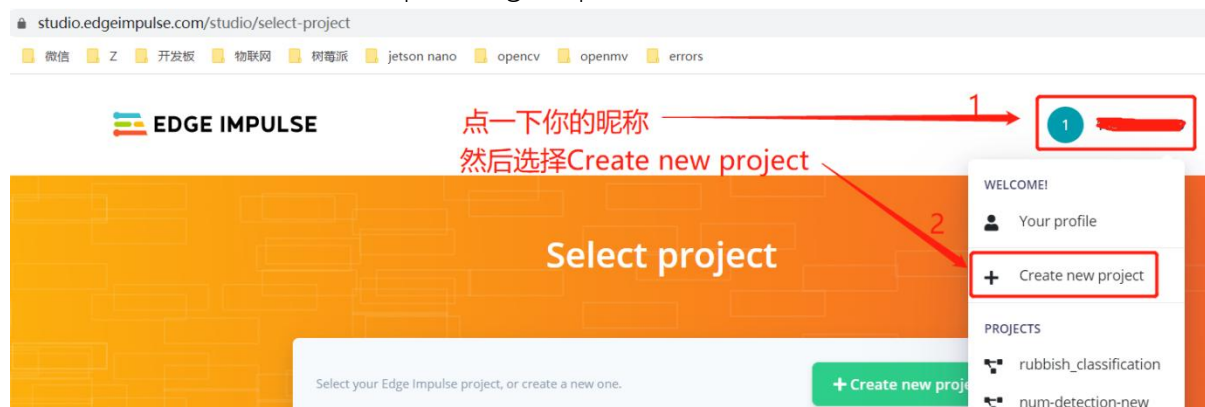
3. 点击运行按钮开始运行程序，然后点一下 0.class，然后把要识别的 0 的图片置于 openmv 摄像头画面之内，点击‘采集数据’按钮来拍照，可以将数字微调角度等然后拍照，拍 200 张左右即可。



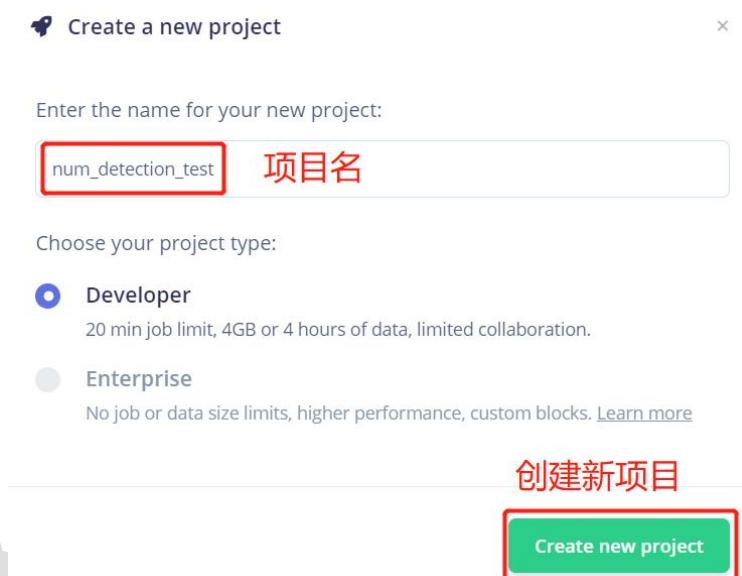
4. 将数字 0 拍完之后，再点一下 1.class，然后将数字 1 置于 openmv 摄像头画面内，点击‘采集数据’进行拍照，同样拍 200 张左右。其他数字同理，拍哪个数字时，就点一下对应的 class 文件夹，然后再将对应数字置于摄像头画面内，点击‘采集数据’按钮进行拍照，拍 200 张左右。

5. 都拍完之后，可以点击停止按钮使代码停止运行。

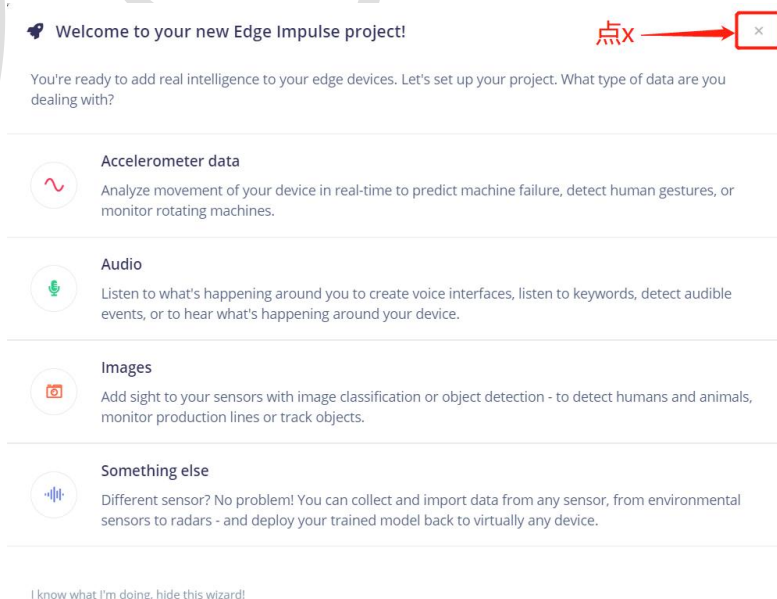
6. 打开浏览器，输入网址 <https://edgeimpulse.com/>，注册并登录进去。



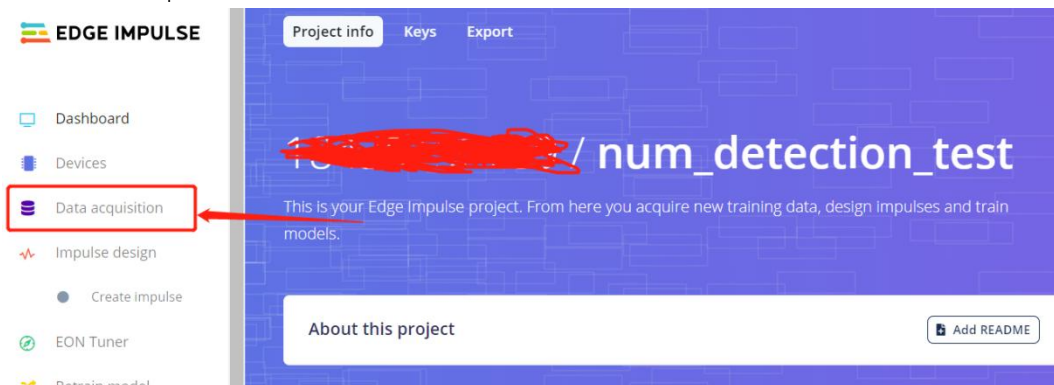
7. 在弹出的界面中为项目起一个名字，然后点击 'Create new project'。



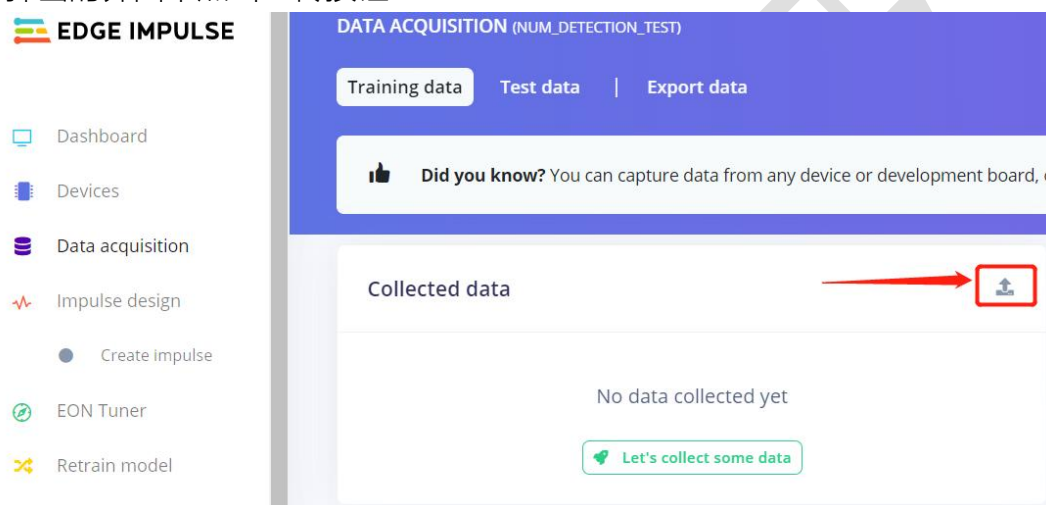
8. 将弹出的如下窗口×掉。



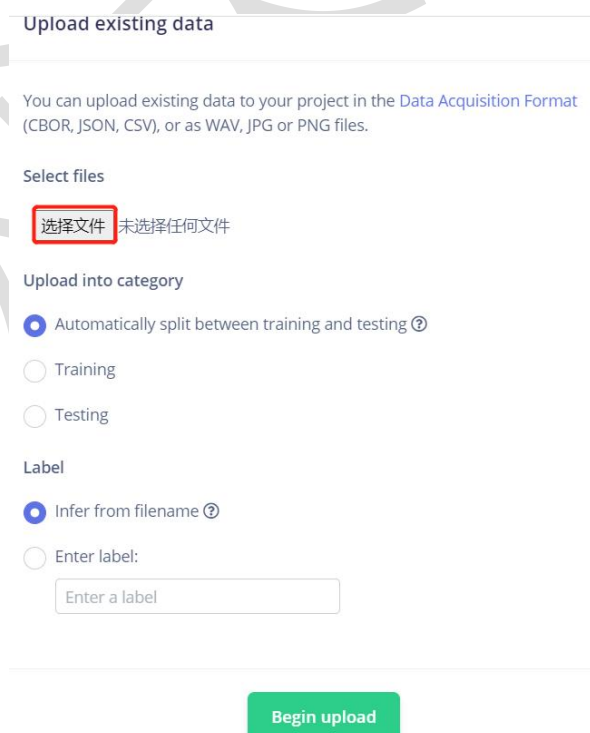
## 9. 点击 Data acquisition



## 10. 在弹出的界面中点击上传按钮

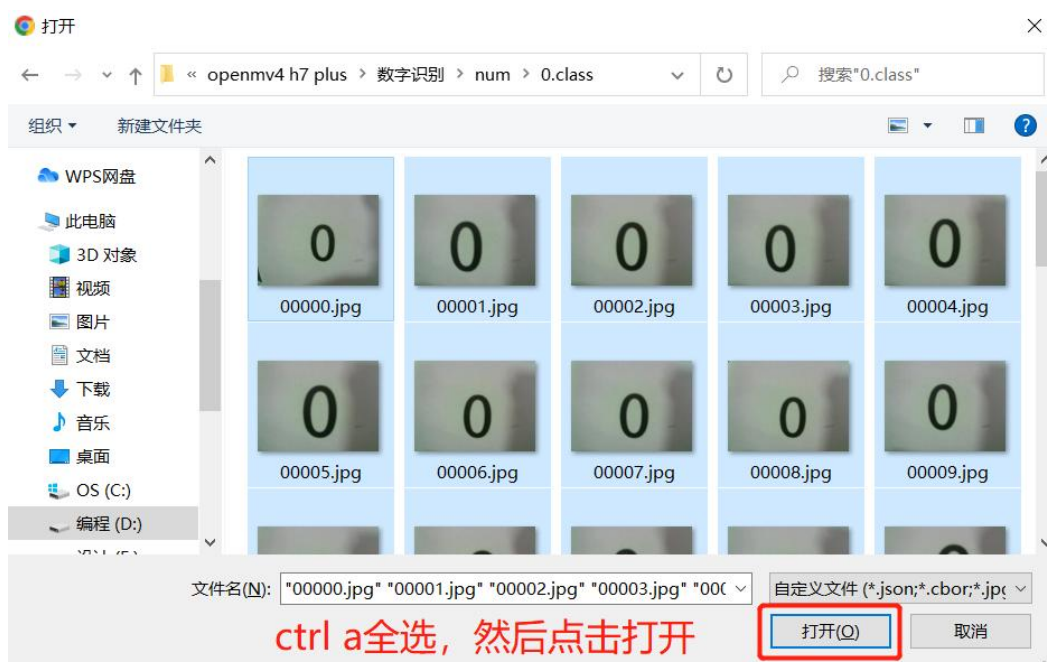


## 11. 点击'选择文件'

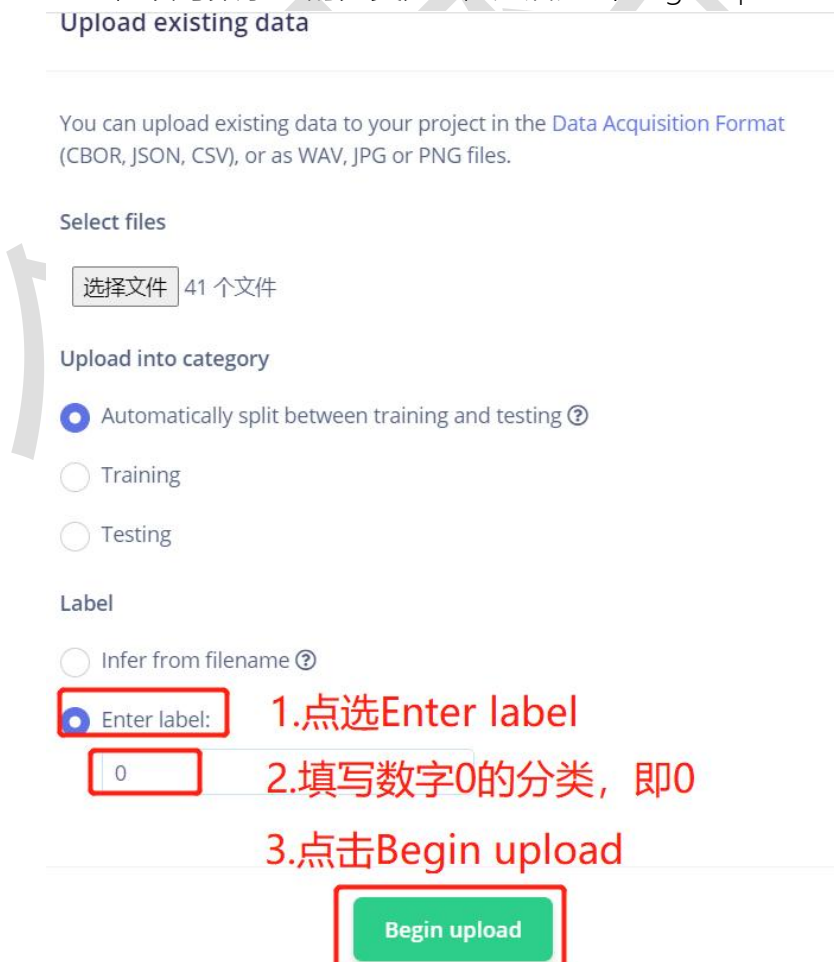




12.选择拍好的那些 0.class 文件夹中的数字 0 的图片，可以按键盘快捷键 ctrl a 全选，然后点击‘打开’



13. 点击 Enter label, 填写数字 0 的分类为 0, 然后点击 Begin upload



14.然后等待右侧上传，如下左图为上传中，右图为上传完成（会提示 Job completed）。

### Upload output

```
[ 9/41] Uploading 00024.jpg OK
[10/41] Uploading 00025.jpg OK
[11/41] Uploading 00026.jpg OK
[12/41] Uploading 00004.jpg OK
[13/41] Uploading 00027.jpg OK
[14/41] Uploading 00028.jpg OK
[15/41] Uploading 00005.jpg OK
[16/41] Uploading 00029.jpg OK
[17/41] Uploading 00006.jpg OK
[18/41] Uploading 00030.jpg OK
[19/41] Uploading 00033.jpg OK
[20/41] Uploading 00031.jpg OK
[21/41] Uploading 00032.jpg OK
[22/41] Uploading 00035.jpg OK
[23/41] Uploading 00034.jpg OK
```

### Upload output

```
[24/41] Uploading 00036.jpg OK
[25/41] Uploading 00037.jpg OK
[26/41] Uploading 00038.jpg OK
[27/41] Uploading 00039.jpg OK
[28/41] Uploading 00040.jpg OK
[29/41] Uploading 00007.jpg OK
[30/41] Uploading 00008.jpg OK
[31/41] Uploading 00009.jpg OK
[32/41] Uploading 00010.jpg OK
[33/41] Uploading 00011.jpg OK
[34/41] Uploading 00012.jpg OK
[35/41] Uploading 00013.jpg OK
[36/41] Uploading 00014.jpg OK
[37/41] Uploading 00018.jpg OK
[38/41] Uploading 00019.jpg OK
[39/41] Uploading 00015.jpg OK
[40/41] Uploading 00017.jpg OK
[41/41] Uploading 00016.jpg OK
```

Done. Files uploaded successful: 41. Files that failed to upload: 0.

Job completed

15.重复第 11 到第 14 步，只不过此次上传 1.class 中的数字 1 的图片，分类处填写 1，然后上传。再次重复 11-14 步将分类 2 上传，以此类推，直到将 0-9 都上传完成。

16.点击 Impulse design

百度 li 微信 Z 开发板 物联网 树莓派 jetson nano opencv openmv errors

### EDGE IMPULSE

- Dashboard
- Devices
- Data acquisition
- Impulse design**
- Create impulse
- EON Tuner
- Retrain model
- Live classification
- Model testing
- Versioning
- Deployment

### Upload existing data

You can upload existing data to your project in the Data Acquisition Format (CBOR, JSON, CSV), or as WAV, JPG or PNG files.

Select files

选择文件 未选择任何文件

Upload into category

☒ Automatically split between training and testing

☐ Training

☐ Testing

Label

☐ Infer from filename

☒ Enter label:

9

Begin upload

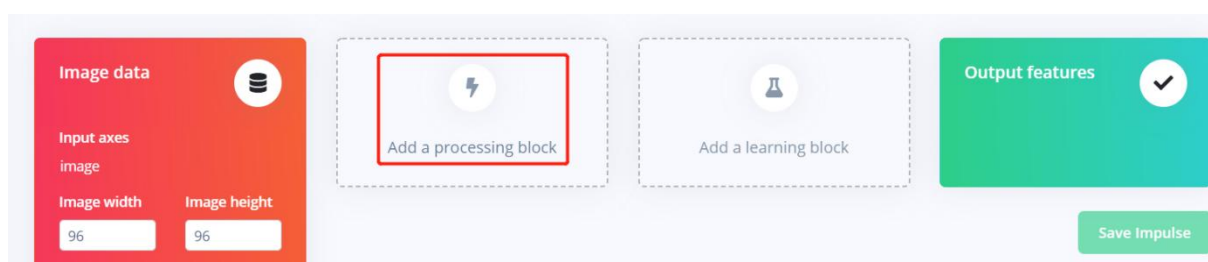
### Upload output

```
[24/41] Uploading 00025.jpg OK
[25/41] Uploading 00023.jpg OK
[26/41] Uploading 00024.jpg OK
[27/41] Uploading 00022.jpg OK
[28/41] Uploading 00028.jpg OK
[29/41] Uploading 00026.jpg OK
[30/41] Uploading 00029.jpg OK
[31/41] Uploading 00031.jpg OK
[32/41] Uploading 00032.jpg OK
[33/41] Uploading 00038.jpg OK
[34/41] Uploading 00030.jpg OK
[35/41] Uploading 00033.jpg OK
[36/41] Uploading 00034.jpg OK
[37/41] Uploading 00036.jpg OK
[38/41] Uploading 00035.jpg OK
[39/41] Uploading 00039.jpg OK
[40/41] Uploading 00037.jpg OK
[41/41] Uploading 00040.jpg OK
```

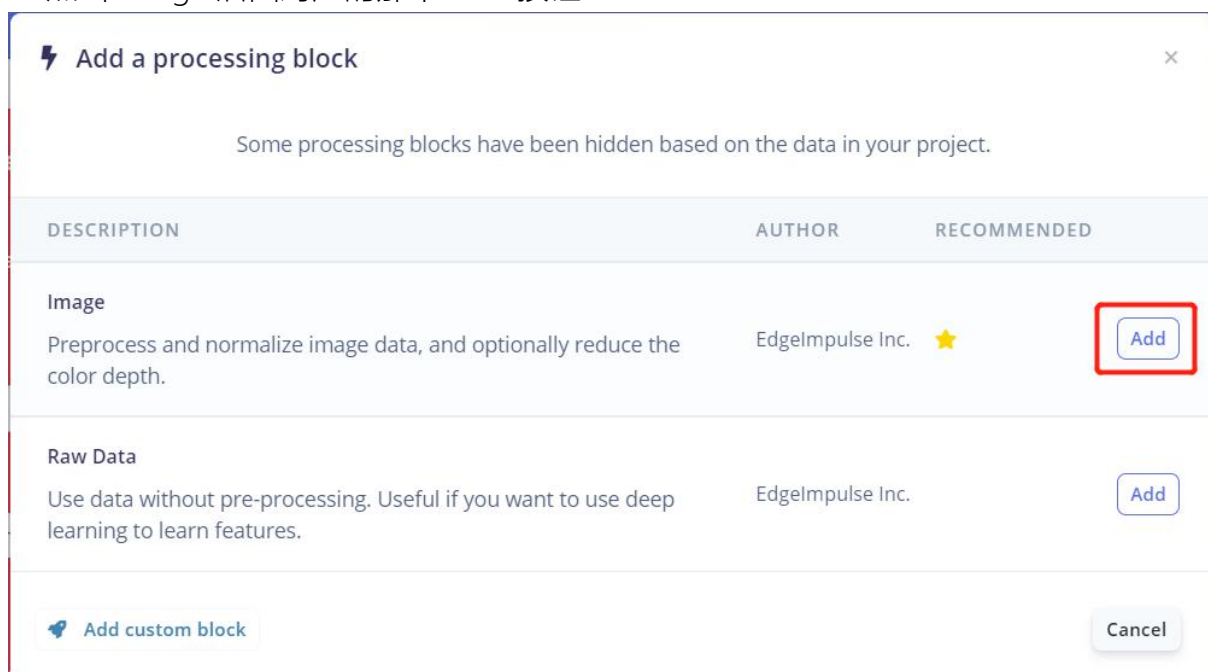
Done. Files uploaded successful

Job completed

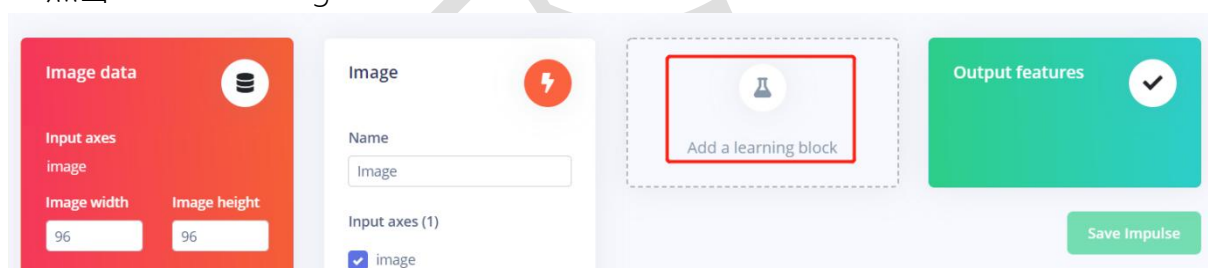
17.在弹出的界面中，点击 Add a processing block



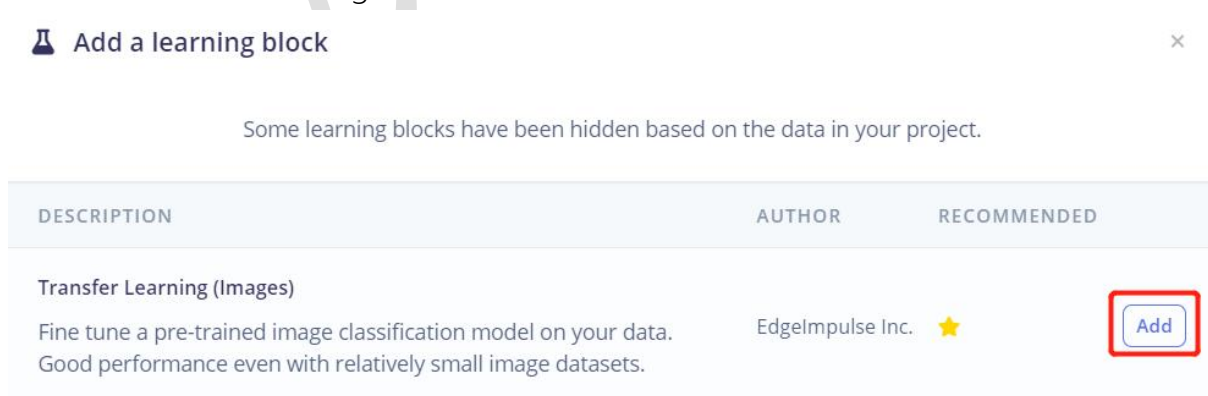
18. 点击 Image 后面对应的那个 Add 按钮



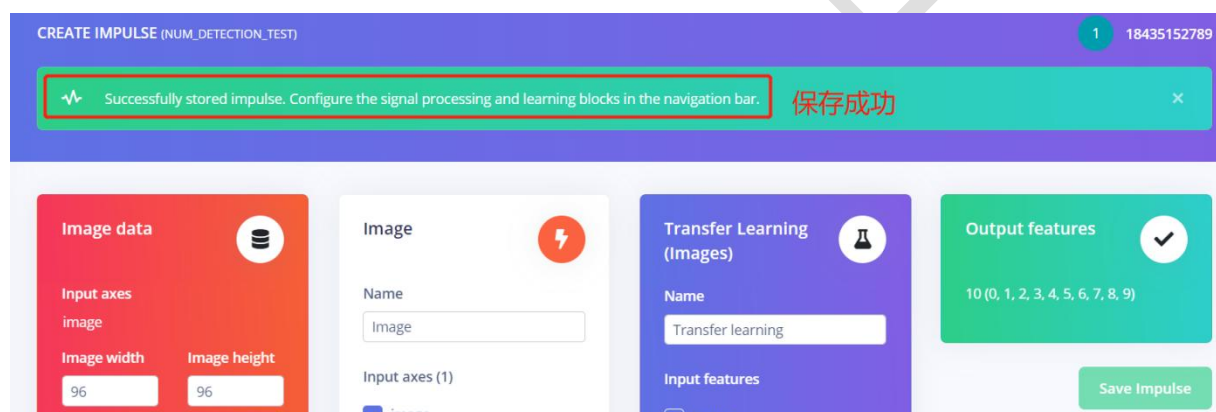
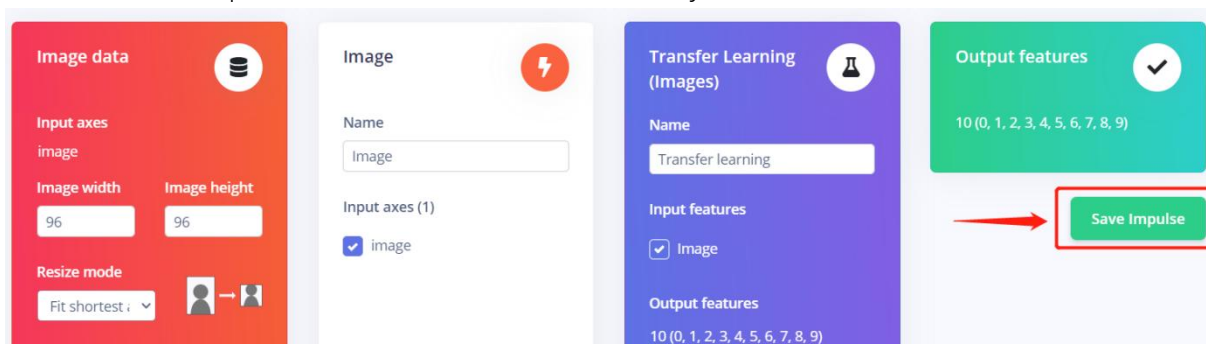
19. 点击 Add a learning block



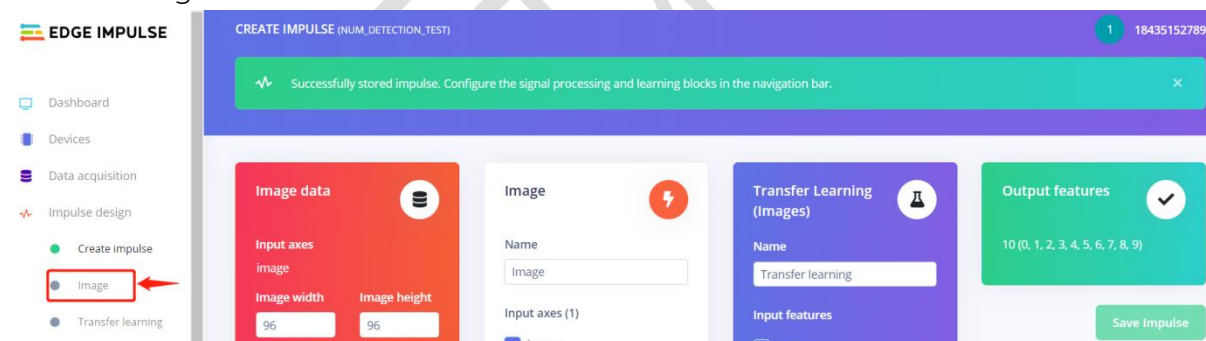
20. 点击 Transfer Learning 后面的那个 Add 按钮



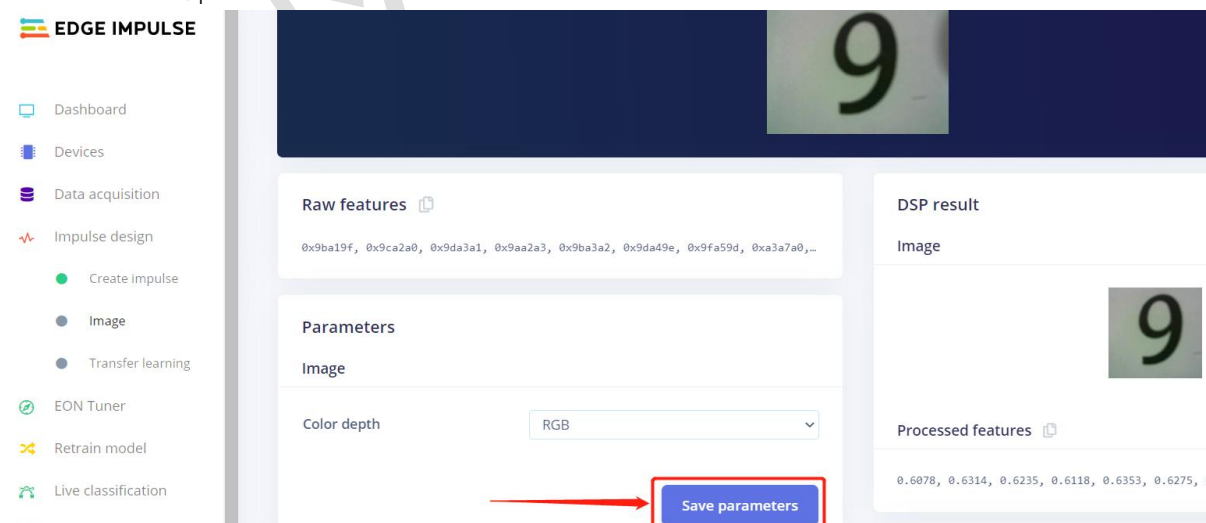
21. 点击 Save Impulse, 然后等待出现 Successfully...则表示保存成功。



22. 点击 Image



23. 点击 Save parameters





24. 点击 Generate features, 然后等待 Generate features 完成(会提示 Job completed)。

**EDGE IMPULSE**

IMAGE (NUM\_DETECTION\_TEST)  
#1 Click to set a description for this version

Parameters **Generate features**

**Training set**

Data in training set 321 items

Classes 10 (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)

**Generate features**

**Feature explorer**

No features generated yet.

**Training set**

Data in training set 321 items

Classes 10 (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)

**Generate features**

**Feature generation output**

```

completed 0 / 500 epochs
completed 50 / 500 epochs
completed 100 / 500 epochs
completed 150 / 500 epochs
completed 200 / 500 epochs
completed 250 / 500 epochs
Still running...
completed 300 / 500 epochs
completed 350 / 500 epochs
completed 400 / 500 epochs
completed 450 / 500 epochs
Tue May 31 10:21:33 2022 Finished embedding
Reducing dimensions for visualizations OK
    
```

**Job completed** **Generate features完成**

**Feature explorer (321 samples)**

X Axis Visualization layer Y Axis Visualization layer Z Axis Visualization layer

Visualization layer 3

Visualization layer 2

Visualization layer 1

此处理分类越清晰，不杂乱，说明效果越好

On-device performance

25. 点击 Transfer learning

**EDGE IMPULSE**

Parameters **Generate features**

**Training set**

Data in training set 321 items

Classes 10 (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)

**Generate features**

**Feature generation output**

```

completed 0 / 500 epochs
completed 50 / 500 epochs
completed 100 / 500 epochs
completed 150 / 500 epochs
completed 200 / 500 epochs
    
```

**Feature explorer (321 samples)**

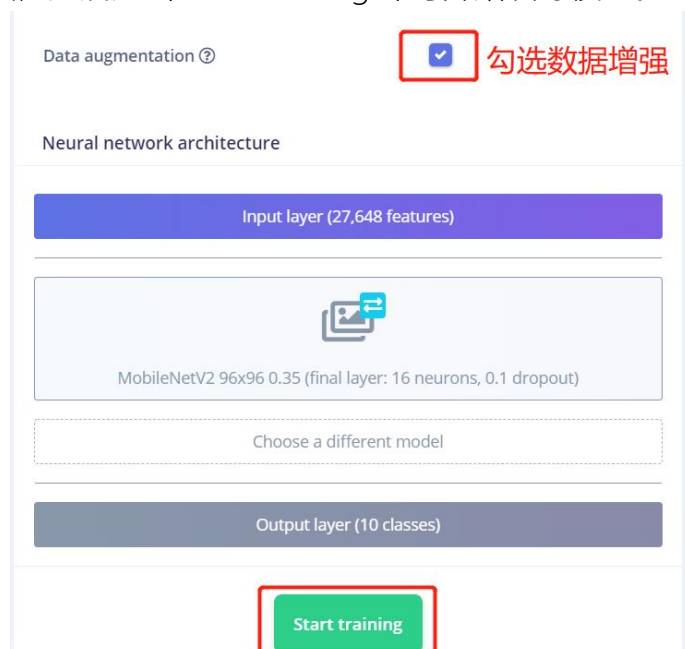
X Axis Visualization layer Y Axis Visualization layer Z Axis Visualization layer

Visualization layer 3

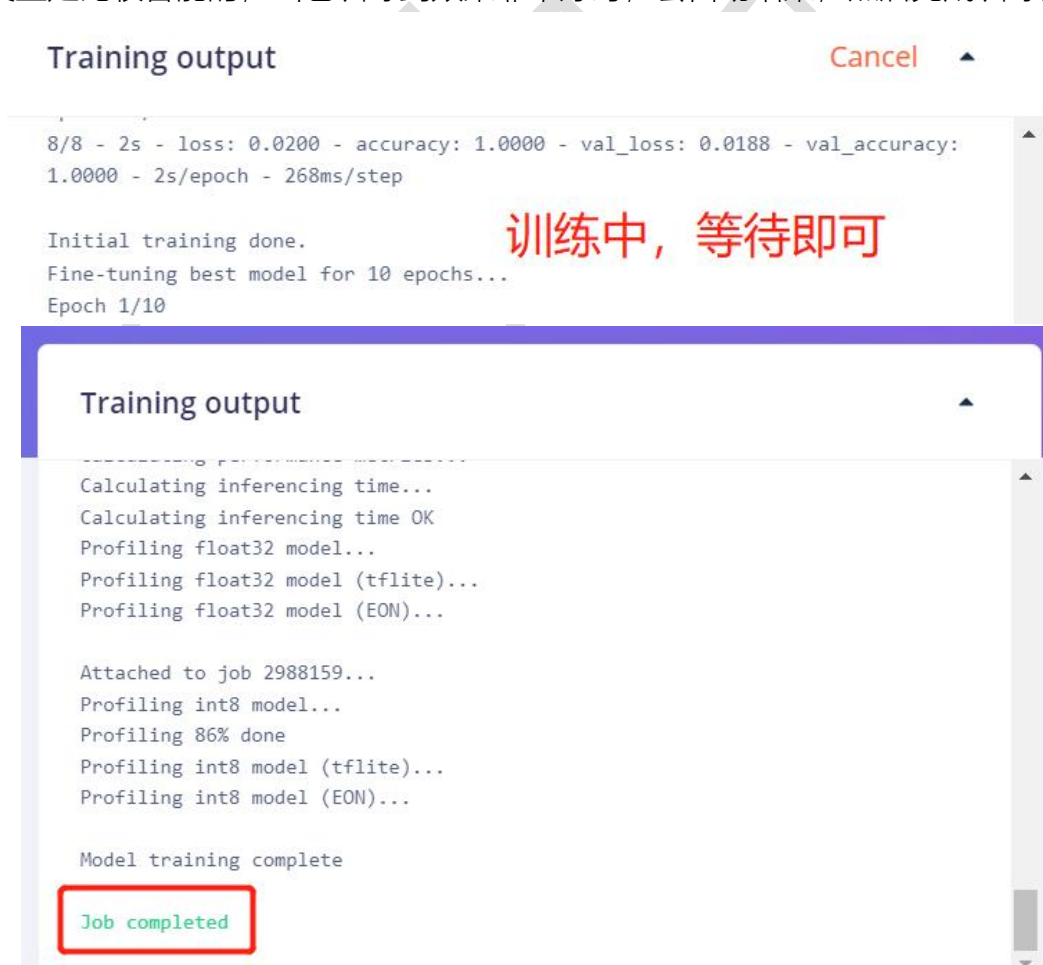
Visualization layer 2

Visualization layer 1

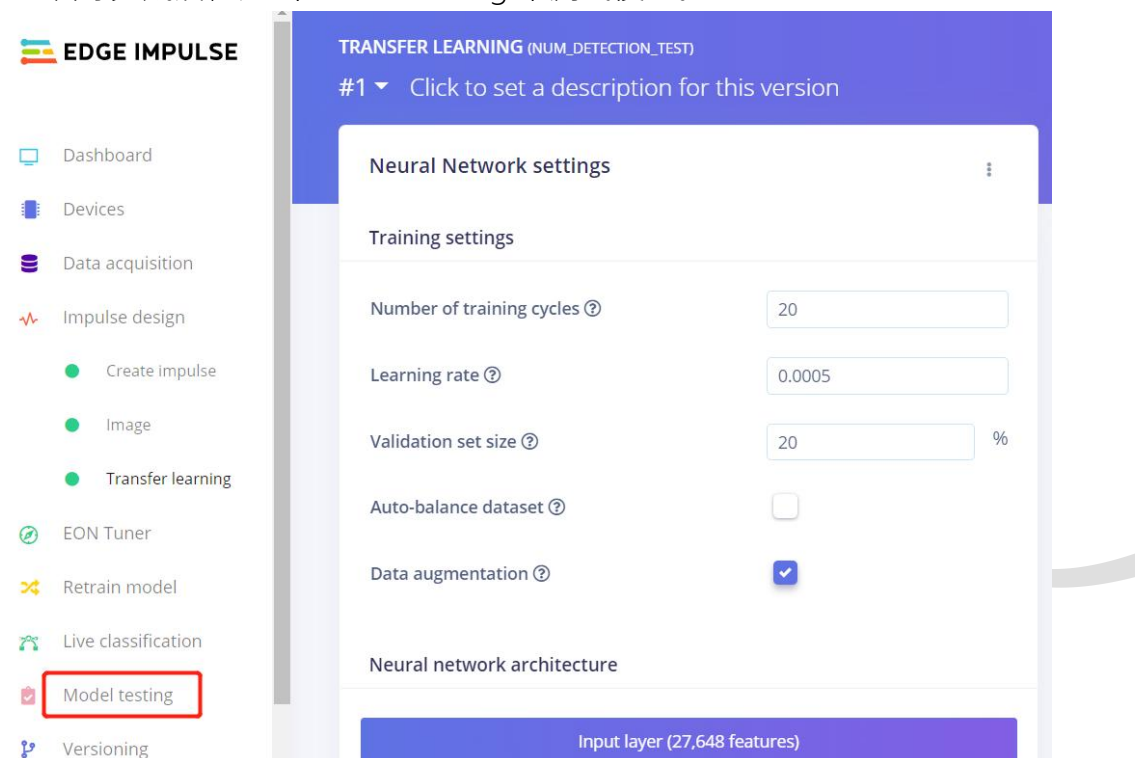
26.如图勾选数据增强，然后点击 Start training 即可开始训练模型。



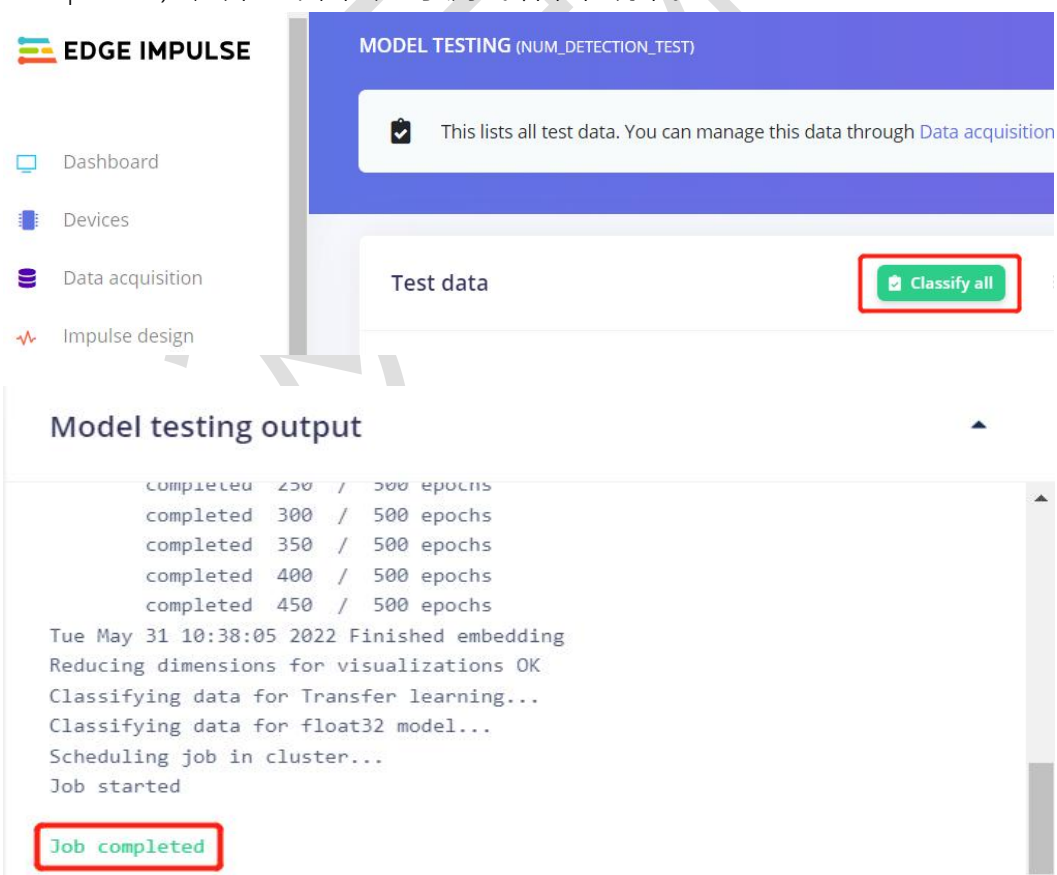
27.然后等待训练完成即可。如下第一张图为训练中，第二张图为训练完成。此网站训练模型是比较智能的，当它训练到效果非常好时，会自动结束，然后完成训练。



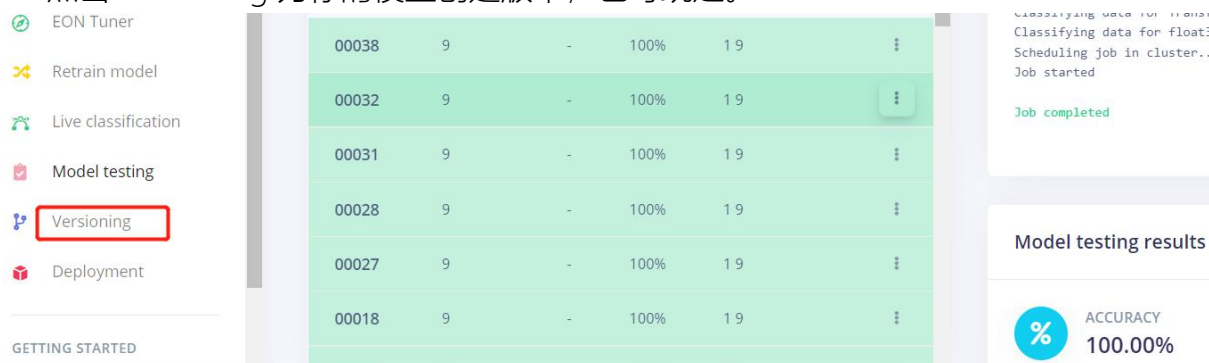
28.训练完成后，点击 Model testing 来测试模型。



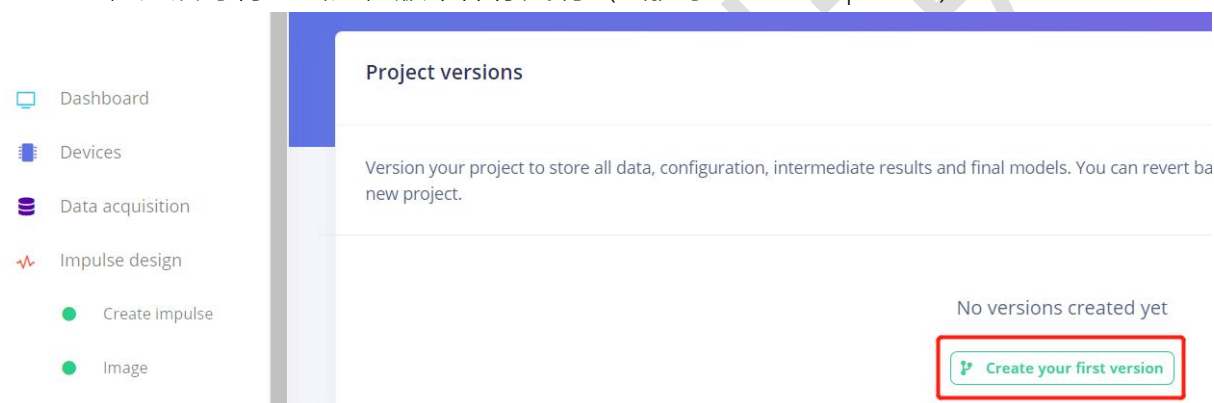
29.点击 Classify all，然后等待验证完成即可。验证完成如下第二张图所示（会提示 Job completed），并且下面会显示测试结果准确率。



30. 点击 Versioning 为你的模型创建版本，也可跳过。



31. 点击 Create your first version, 然后在弹出的界面中填写版本描述，然后点击 Store version, 然后等待一会儿，版本保存成功（会提示 Job completed）



Store current project version

Store current project version

New project version: 1 (版本号)

Description: num-detection-V1.0 (版本描述)

Storage location: Edge Impulse hosted

Make version public: ☐

保存版本 (Store version)

## Store current project version

完成后，点x

x

```
Scheduling job in cluster...
Job started
[6/9] Waiting for S3 mount to become available...
[6/9] Waiting for S3 mount to become available...
[6/9] Copying block data...
[6/9] Still copying...
[6/9] Copying block data OK

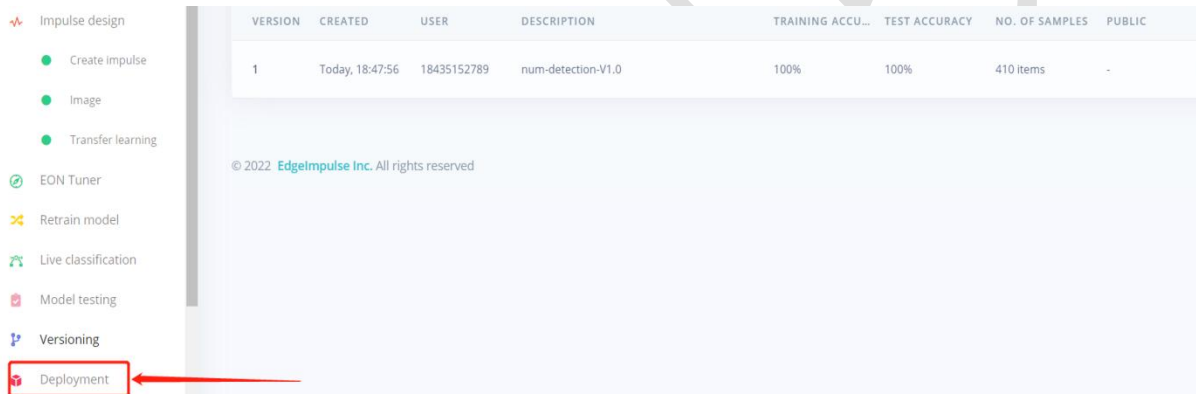
[7/9] Archiving files...
[7/9] Still archiving...
[7/9] Still archiving...
[7/9] Archiving files OK

[8/9] Uploading version...
[8/9] Still uploading...
[8/9] Uploading version OK

[9/9] Storing version 1...
[9/9] Storing version 1 OK
```

Job completed

32.在版本保存完成后，点x号，然后点击左侧的 Deployment

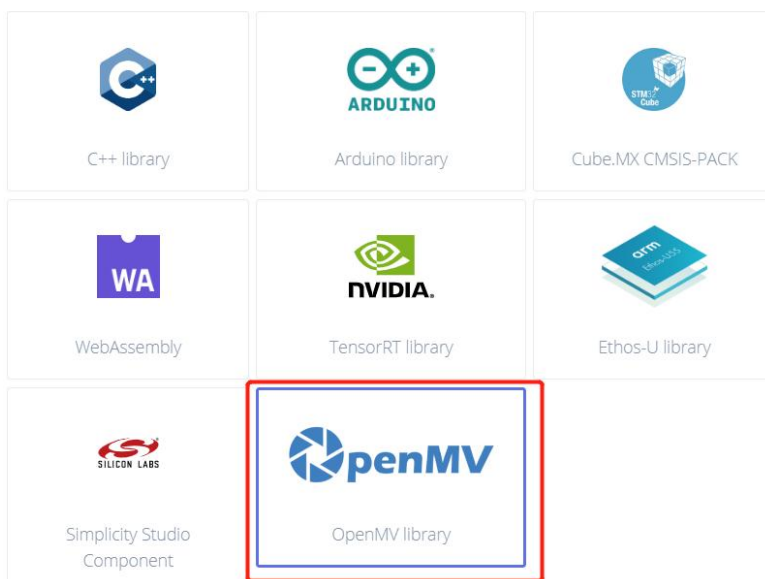


33.点击 OpenMV library

## EDGE IMPULSE

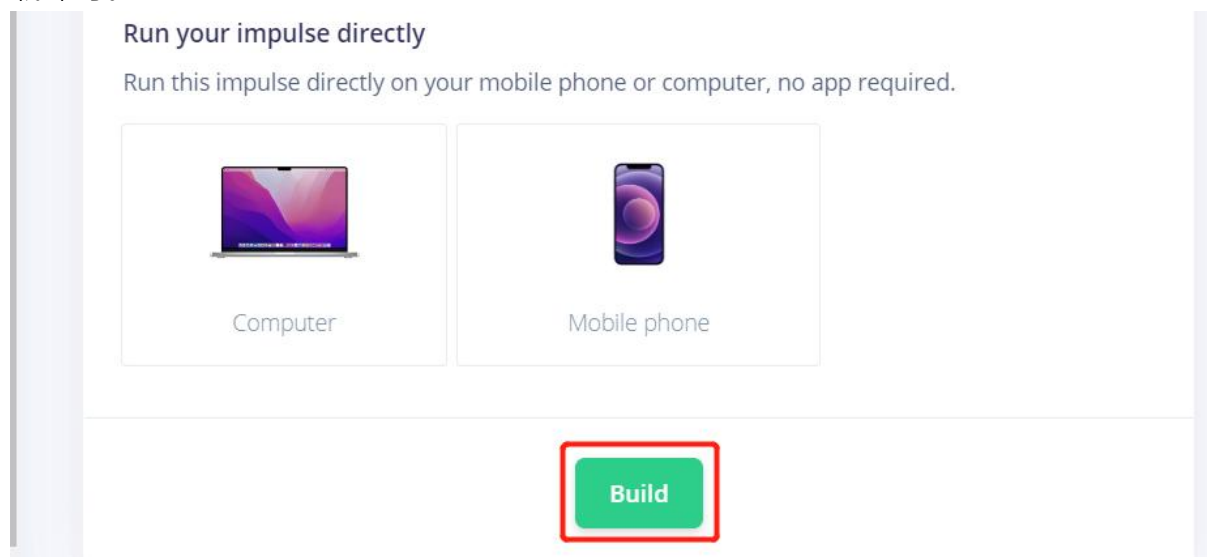
Turn your impulse into optimized source code that you can run on any device.

- Dashboard
- Devices
- Data acquisition
- Impulse design
  - Create impulse
  - Image
  - Transfer learning
- EON Tuner
- Retrain model
- Live classification



34.然后点击 Build，等待几秒钟，它会让你选择下载位置，我们选一个下载位置进行下

载即可。



35.将下载的压缩包解压，然后可以看到共有 3 个文件，其中：trained.tflite 为训练好的神经网络模型文件，labels.txt 是配套的分类文件，ei\_image\_classification.py 为配套代码，模型和分类文件最好不要改名（在代码中会用到，不宜改名，如果改名的话，在代码中也要做相应修改），代码可以改名

ei-num\_detection\_test-openmv-v2

| 名称                         | 修改日期            | 类型               | 大小     |
|----------------------------|-----------------|------------------|--------|
| ei_image_classification.py | 2022/5/31 10:58 | "mPython" "File" | 2 KB   |
| labels.txt                 | 2022/5/31 10:58 | 文本文档             | 1 KB   |
| trained.tflite             | 2022/5/31 10:58 | TFLITE 文件        | 626 KB |

注意：你训练好的这个模型文件 trained.tflite 和分类文件 labels.txt 是配套我们提供的那个代码文件 num\_detection.py 的，可以将你训练的这个模型文件 trained.tflite 和分类文件 labels.txt 复制粘贴到 openmv 的盘符中，然后在 ide 中运行 num\_detection.py 来使用的。

### 【本课小结】：

注意：

1. 使用训练神经网络模型的方法实现数字识别只适用于 openmv h7 plus。
2. 在我们的资料中的对应源码例程中有提供的训练好的模型文件 trained.tflite 和配套文件 labels.txt，你可以直接将模型文件 trained.tflite 和配套文件 labels.txt 复制粘贴到 openmv 的盘符中，然后在 ide 中运行代码 num\_detection.py 来使用即可，可以用提供的配套的测试图片来进行测试。
3. 你也可以自己训练想要识别的一些数字的神经网络模型来使用。



## 第 14 课 口罩识别（只适用于 openmv4 h7 plus）

### 【本课目标】：

本课学习通过训练神经网络模型实现识别口罩的功能

### 【课前准备】：

OpenMV4 H7 PLUS、USB 数据线

### 【本课内容】：

注意：

1. 使用训练神经网络模型的方法实现口罩识别只适用于 openmv h7 plus。
2. 在我们的资料中的对应源码例程中有提供的训练好的模型文件 trained.tflite 和配套文件 labels.txt，你可以直接将模型文件 trained.tflite 和配套文件 labels.txt 复制粘贴到 openmv 的盘符中，然后在 ide 中运行代码 mask\_detection.py 来使用即可。
3. 你也可以自己训练神经网络模型来使用，具体训练方法在上一节数字识别中已经讲过，可以参考，不同的地方主要就是新类文件夹的名称是 face 和 mask，然后在网址中上传文件时的分类名需要对应为 face 和 mask，其他步骤都一样。

### 一、代码

```
1. # TensorFlow 口罩识别
2.
3. import sensor, image, time, os, tf, uos, gc
4.
5. net = None
6. labels = None
7.
8. # 初始化函数
9. def mask_init_setup():
10.     global sensor, net, labels, clock # 设置为全局变量
11.     sensor.reset() # 初始化感光元件
12.     sensor.set_pixformat(sensor.RGB565) # 设置图像格式为彩色
13.     sensor.set_framesize(sensor.QVGA) # 设置图像大小为 QVGA (320x240)
14.     sensor.set_windowing((240, 240)) # 设置 window 窗口大小为 240x240
15.     sensor.skip_frames(10) # 跳过一些帧，使以上设置生效
16.
17.     try:
18.         # 加载模型文件
19.         net = tf.load("trained.tflite", load_to_fb=uos.stat('trained.tflite')[6] > (gc.mem_free() - (64*1024)))
20.     except Exception as e:
21.         print(e)
22.         raise Exception('Failed to load "trained.tflite", did you copy the .tflite and labels.txt file onto the mass-storage device? (' + str(e) + ')')
23.
24.     try:
```



```
25.         labels = [line.rstrip('\n') for line in open("labels.txt")]
26.     except Exception as e:
27.         raise Exception('Failed to load "labels.txt", did you copy the .tflite and labels.txt file onto the
    mass-storage device? (' + str(e) + ')')
28.
29.     clock = time.clock()                # 创建时钟对象
30.
31. # 主函数
32. def mask_main():
33.     while 1:
34.         clock.tick()                    # 更新 FPS 帧率时钟
35.         img = sensor.snapshot()         # 拍一张照片并返回图像
36.
37.         # default settings just do one detection... change them to search the image...
38.         for obj in net.classify(img, min_scale=1.0, scale_mul=0.8, x_overlap=0.5, y_overlap=0.5):
39.             #print("*****\nPredictions at [x=%d,y=%d,w=%d,h=%d]" % obj.rect())
40.             img.draw_rectangle(obj.rect())
41.             # 将分类和对应的相似度以列表套元组形式返回
42.             predictions_list = list(zip(labels, obj.output()))
43.             # 打印分类和与图像中检测到的物体的对应的相似度
44.             #for i in range(len(predictions_list)):
45.                 #print("%s = %f" % (predictions_list[i][0], predictions_list[i][1]))
46.             if max(obj.output()) > 0.9: # 当识别到的最大的相似度大于 0.9 时才认为是识别到了
47.                 # 将识别到的类别赋值给 result
48.                 result = labels[obj.output().index(max(obj.output()))]
49.                 if result == 'mask':
50.                     # 打印识别到的类别
51.                     print('检测到已戴好口罩')
52.                 elif result == 'face':
53.                     print('检测到未佩戴口罩, 请佩戴好口罩')
54.
55.             #print(clock.fps(), "fps") # 打印帧率
56.
57. # 程序入口
58. if __name__ == '__main__':
59.     mask_init_setup() # 执行初始化函数
60.     try:
61.         mask_main() # 执行主函数
62.     except:
63.         pass
```

## 二、代码使用方法

将 labels.txt 和 trained.tflite 复制粘贴到 openmv 的盘符中, 然后在 openmv ide 中打开 mask\_detection.py 运行使用即可。

## 三、运行效果

代码运行后, openmv 会将识别到的结果在串行终端窗口打印出来。

## 四、知识点

训练神经网络模型的方法在上一节数字识别课程中已经讲过, 此处不再赘述。

关于 tensorflow 神经网络模型的使用可以参考官方文档：

<https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.tflite/ml/>

### 【本课小结】：

注意：

1. 使用训练神经网络模型的方法实现口罩识别只适用于 openmv h7 plus。
2. 在我们的资料中的对应源码例程中有提供的训练好的模型文件 trained.tflite 和配套文件 labels.txt，你可以直接将模型文件 trained.tflite 和配套文件 labels.txt 复制粘贴到 openmv 的盘符中，然后在 ide 中运行代码 mask\_detection.py 来使用即可。
3. 你也可以自己训练神经网络模型来使用，具体训练方法在上一节数字识别中已经讲过，可以参考，不同的地方主要就是新类文件夹的名称是 face 和 mask，然后在网址中上传文件时的分类名需要对应为 face 和 mask，其他步骤都一样。

## 第 15 课 垃圾分类（只适用于 openmv4 h7 plus）

### 【课程目标】：

本课学习通过训练神经网络模型实现垃圾分类的功能

### 【课前准备】：

OpenMV4 H7 PLUS、USB 数据线

### 【本课内容】：

注意：

1. 使用训练神经网络模型的方法实现垃圾分类只适用于 openmv h7 plus。
2. 在我们的资料中的对应源码例程中有提供的训练好的模型文件 trained.tflite 和配套文件 labels.txt，你可以直接将模型文件 trained.tflite 和配套文件 labels.txt 复制粘贴到 openmv 的盘符中，然后在 ide 中运行代码 rubbish\_classification.py 来使用即可。
3. 你也可以自己训练神经网络模型来使用，具体训练方法在数字识别中已经讲过，可以参考，不同的地方主要就是新类文件夹的名称需要是你要识别的垃圾的名称，然后在网址中上传文件时的分类名需要是你要识别的垃圾的名称，其他步骤都一样。

### 一、代码

```
1.  # TensorFlow 垃圾分类
2.
3.  import sensor, image, time, os, tf, uos, gc
4.
5.  net = None
6.  labels = None
7.
8.  # 初始化函数
9.  def rubbish_init_setup():
10.     global sensor, net, labels, clock    # 设置为全局变量
11.     sensor.reset()                        # 初始化感光元件
12.     sensor.set_pixformat(sensor.GRAYSCALE) # 设置图像格式为灰度
13.     sensor.set_framesize(sensor.QVGA)     # 设置图像大小为 QVGA (320x240)
14.     sensor.set_windowing((240, 240))     # 设置窗口大小为 240x240
15.     sensor.skip_frames(10)                # 跳过一些帧，使以上设置生效
16.
17.     try:
18.         # 加载模型文件
```

```

19.         net = tf.load("trained.tflite", load_to_fb=uos.stat('trained.tflite')[6] > (gc.mem_free()
    - (64*1024)))
20.     except Exception as e:
21.         print(e)
22.         raise Exception('Failed to load "trained.tflite", did you copy the .tflite and labels.txt
    file onto the mass-storage device? (' + str(e) + ')')
23.
24.     try:
25.         labels = [line.rstrip('\n') for line in open("labels.txt")]
26.     except Exception as e:
27.         raise Exception('Failed to load "labels.txt", did you copy the .tflite and labels.txt file
    onto the mass-storage device? (' + str(e) + ')')
28.
29.     clock = time.clock()          # 创建时钟对象
30.
31. # 主函数
32. def rubbish_main():
33.     while 1:
34.         clock.tick()              # 更新 FPS 帧率时钟
35.         img = sensor.snapshot()    # 拍一张照片并返回图像
36.
37.         # default settings just do one detection... change them to search the image...
38.         for obj in net.classify(img, min_scale=1.0, scale_mul=0.8, x_overlap=0.5, y_overlap=0.5):
39.
40.             #print("*****\nPredictions at [x=%d,y=%d,w=%d,h=%d]" % obj.rect())
41.             # 将分类和对应的相似度以列表套元组形式返回
42.             predictions_list = list(zip(labels, obj.output()))
43.             # 打印分类和与图像中检测到的物体的对应的相似度
44.             #for i in range(len(predictions_list)):
45.                 #print("%s = %f" % (predictions_list[i][0], predictions_list[i][1]))
46.             if max(obj.output()) > 0.7: # 当识别到的最大的相似度大于 0.7 时才认为是识别到了垃圾
47.                 # 将识别到的垃圾名称赋值给 rubbish_name
48.                 rubbish_name = labels[obj.output().index(max(obj.output()))]
49.                 # 打印识别到的垃圾名称
50.                 print('识别到的垃圾是 %s' % rubbish_name)
51.
52.             #print(clock.fps(), "fps") # 打印帧率
53.
54. # 程序入口
55. if __name__ == '__main__':
56.     rubbish_init_setup() # 执行初始化函数
57.     try:
58.         rubbish_main()   # 执行主函数

```

```
59.         except:
```

```
60.             pass
```

## 二、代码使用方法

将 labels.txt 和 trained.tflite 复制粘贴到 openmv 的盘符中，然后在 openmv ide 中打开 rubbish\_classification.py 运行使用即可。

## 三、运行效果

代码运行后，openmv 会将识别到的结果在串行终端窗口打印出来。

我们示例源码程序使用的模型文件只包含了以下四种垃圾，可以使用我们的测试图片来进行测试。

(只适用于openmv4 h7 plus) > 测试图片 > 🔍 搜索"测试图"

| 名称            | 修改日期            | 类型  |
|---------------|-----------------|-----|
| 📁 电池dianchi   | 2022/5/26 17:19 | 文件夹 |
| 📁 螺丝刀luosidao | 2022/5/26 17:27 | 文件夹 |
| 📁 套筒taotong   | 2022/5/26 17:34 | 文件夹 |
| 📁 纸杯zhibei    | 2022/5/26 17:43 | 文件夹 |

## 四、知识点

训练神经网络模型的方法在上一节数字识别课程中已经讲过，此处不再赘述。

关于 tensorflow 神经网络模型的使用可以参考官方文档：

<https://docs.singtown.com/micropython/zh/latest/openmvcam/library/openmv.tf.html#>

### 【本课小结】：

注意：

1. 使用训练神经网络模型的方法实现垃圾分类只适用于 openmv h7 plus。
2. 在我们的资料中的对应源码例程中有提供的训练好的模型文件

trained.tflite 和配套文件 labels.txt，你可以直接将模型文件 trained.tflite 和配套文件 labels.txt 复制粘贴到 openmv 的盘符中，然后在 ide 中运行代码 rubbish\_classification.py 来使用即可。

3. 你也可以自己训练神经网络模型来使用，具体训练方法在数字识别中已经讲过，可以参考，不同的地方主要就是新类文件夹的名称需要是你识别的垃圾的名称，然后在网址中上传文件时的分类名需要是你识别的垃圾的名称，其他步骤都一样。

众灵科技